

Bài 7: LẬP TRÌNH PYTHON CHO PHÂN TÍCH DỮ LIỆU VÀ HỌC MÁY (Phân tích và xử lý dữ liệu với Pandas - 02)

AI Academy Vietnam

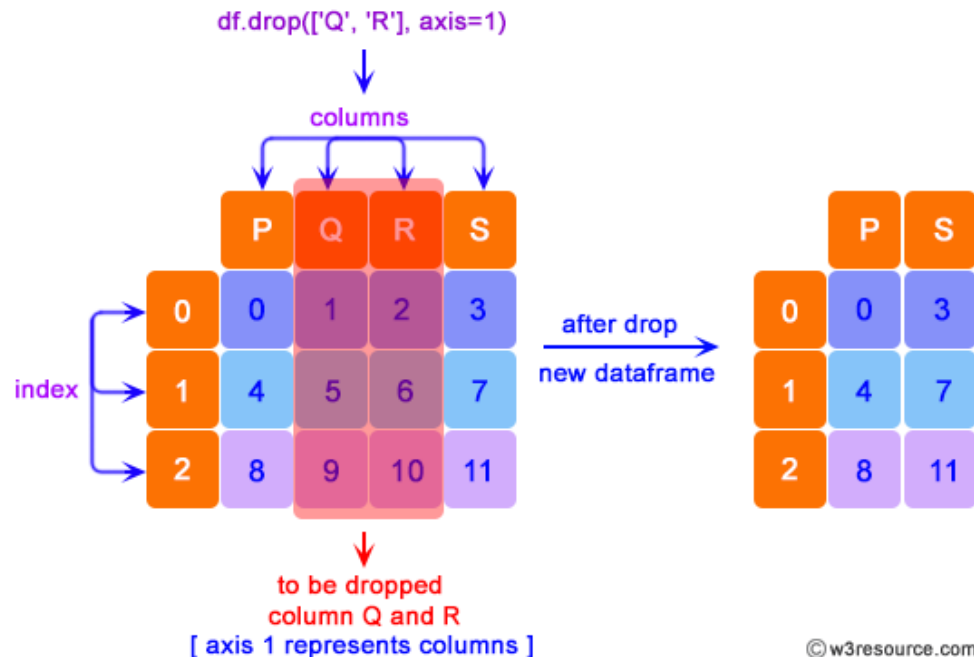
Nội dung bài 7

1. Xóa cột/hàng trong Dataframe
2. Sắp xếp trong DataFrame (Sort)
3. Áp dụng hàm cho các phần tử trong DataFrame (Apply)
4. Nhóm dữ liệu (Group)
5. Trộn các DataFrame (Merging)
6. Ghép nối các DataFrame (Concatenating)
7. Xử lý hàng trùng lặp (Duplicate rows)
8. Phát hiện và xử lý giá trị khuyết thiếu (Missing)
9. Phát hiện và xử lý giá trị ngoại lai (Outliers)
10. Phân tích dữ liệu bán hàng (Tiếp cận từ bài toán thực tế)

1. Xóa cột/hàng trong DataFrame

1.1 Xóa cột

- `df.drop([column_name], axis=1 | 'columns', inplace=True | False)`: Để xóa 1 cột hoặc nhiều cột trong một DataFrame.
- Lưu ý khi sử dụng tham số `inplace = True` → Áp dụng thay đổi cho chính DataFrame hiện tại



```
1 #Xử dụng .drop(axis=1/columns) để xóa cột
2 #Xóa một số cột trong df_loan
3 df_loan1 = df_loan.drop(['annual_inc',
4                           'dti', 'delinq_2yrs',
5                           'revol_util',
6                           'longest_credit_length',
7                           'verification_status'], axis=1)
8 df_loan1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 163987 entries, 0 to 163986
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   loan_amnt             163987 non-null  int64
1   term                  163987 non-null  object
2   int_rate              163987 non-null  float64
3   emp_length            158183 non-null  float64
4   home_ownership        163987 non-null  object
5   purpose               163987 non-null  object
6   addr_state            163987 non-null  object
7   total_acc             163958 non-null  float64
8   bad_loan              163987 non-null  int64
dtypes: float64(3), int64(2), object(4)
memory usage: 11.3+ MB
```

1.1 Xóa cột

- `df.drop(df.columns[index], axis=1 | columns)`: Để xóa 1 cột hoặc nhiều cột trong một DataFrame trong trường hợp DataFrame không có tên cột, sử dụng chỉ số cột.

```
1 #Xóa cột trong một DataFrame sử dụng chỉ số cột
2 df_loan2 = df_loan.drop(df_loan.columns[[5,8,9,10,13,14]],
3                          axis='columns')
4 df_loan2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 163987 entries, 0 to 163986
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   loan_amnt             163987 non-null  int64
1   term                  163987 non-null  object
2   int_rate              163987 non-null  float64
3   emp_length            158183 non-null  float64
4   home_ownership        163987 non-null  object
5   purpose               163987 non-null  object
6   addr_state            163987 non-null  object
7   total_acc             163958 non-null  float64
8   bad_loan              163987 non-null  int64
dtypes: float64(3), int64(2), object(4)
memory usage: 11.3+ MB
```

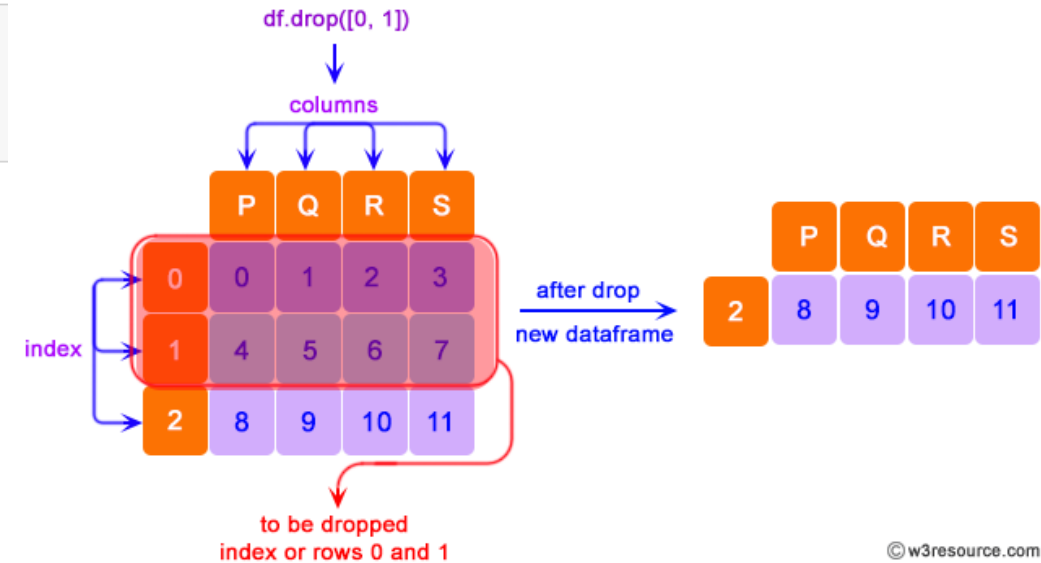
1.2 Xóa hàng

Xóa hàng theo index:

- **df.drop([index rows], axis=0):** Để xóa 1 hàng hoặc nhiều hàng trong một DataFrame theo index của hàng.
- Tham số axis = 0 (Default)

```
1 #Xóa hàng trong một DataFrame
2 #Xóa hàng có index: 3,9
3 df_loan2.drop([3,9],inplace=True)
4 df_loan2.head(10)
```

	loan_amnt	term	int_rate	emp_length	home_ownership	purpose	addr_state	total_acc	bad_loan
0	5000	36 months	10.65	10.0	RENT	credit_card	AZ	9.0	0
1	2500	60 months	15.27	0.0	RENT	car	GA	4.0	1
2	2400	36 months	15.96	10.0	RENT	small_business	IL	10.0	0
4	5000	36 months	7.90	3.0	RENT	wedding	AZ	12.0	0
5	3000	36 months	18.64	9.0	RENT	car	CA	4.0	0
6	5600	60 months	21.28	4.0	OWN	small_business	CA	13.0	1
7	5375	60 months	12.69	0.0	RENT	other	TX	3.0	1
8	6500	60 months	14.65	5.0	OWN	debt_consolidation	AZ	23.0	0
10	9000	36 months	13.49	0.0	RENT	debt_consolidation	VA	9.0	1
11	3000	36 months	9.91	3.0	RENT	credit_card	IL	11.0	0



1.2 Xóa hàng

Xóa hàng theo điều kiện (filter data):

```
1 #Loại bỏ tất cả các dòng dữ liệu có addr_state = CA
2 df_loan3 = df_loan1[df_loan1.addr_state!='CA']
3 df_loan3
```

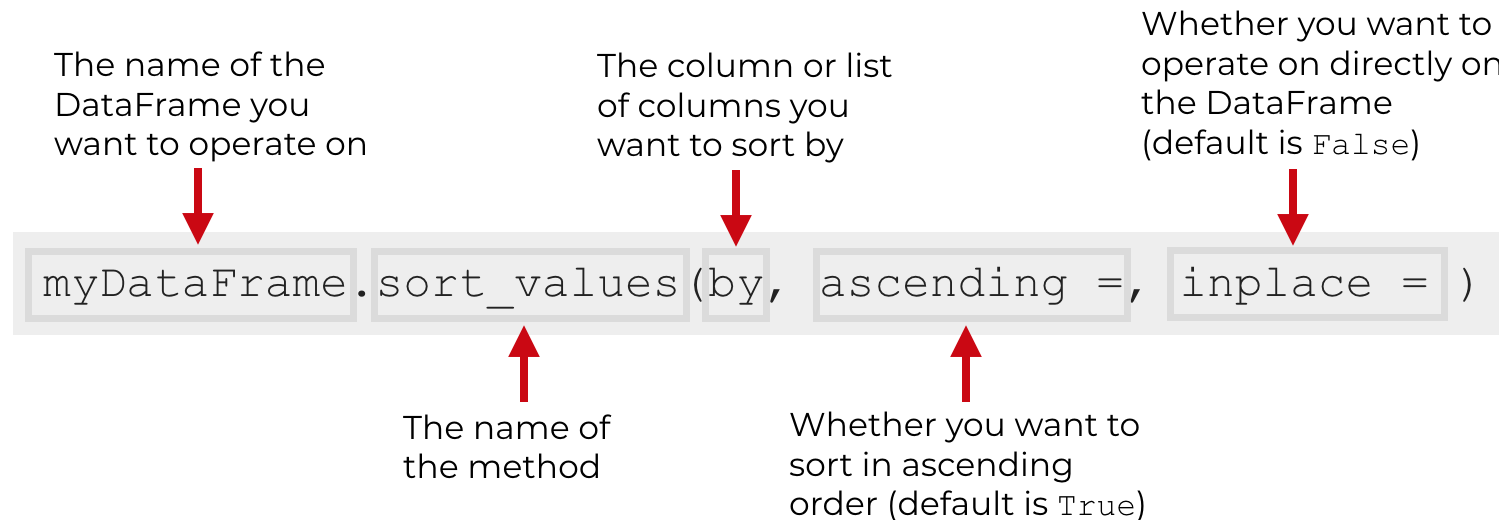
	loan_amnt	term	int_rate	emp_length	home_ownership	purpose	addr_state	total_acc	bad_loan
0	5000	36 months	10.65	10.0	RENT	credit_card	AZ	9.0	0
1	2500	60 months	15.27	0.0	RENT	car	GA	4.0	1
2	2400	36 months	15.96	10.0	RENT	small_business	IL	10.0	0
4	5000	36 months	7.90	3.0	RENT	wedding	AZ	12.0	0
7	5375	60 months	12.69	0.0	RENT	other	TX	3.0	1
...	...	...	...	...	...	...	...	...	...
163982	15000	60 months	12.39	3.0	MORTGAGE	credit_card	OK	34.0	0
163983	20000	36 months	14.99	10.0	OWN	home_improvement	VA	18.0	0
163984	12825	36 months	17.14	6.0	MORTGAGE	debt_consolidation	TX	24.0	0
163985	27650	60 months	21.99	0.0	RENT	credit_card	NY	20.0	0
163986	17000	60 months	15.99	10.0	MORTGAGE	debt_consolidation	PA	28.0	0

135285 rows × 9 columns

2. Sắp xếp dữ liệu trong DataFrame

3. Sắp xếp dữ liệu

- **df.sort_values()**: sắp xếp dữ liệu trong DataFrame theo giá trị của các cột, tăng dần (**ascending = True**)-default hoặc giảm dần (**ascending=False**)
- Lưu ý khi sử dụng tham số **inplace = True**



- **df.sort_index()**: sắp xếp dữ liệu trong DataFrame theo index

3. Sắp xếp dữ liệu

- Ví dụ:

	Name	Age	Score
0	Alisa	26	89
1	Bobby	27	87
2	Cathrine	25	67
3	Madonna	24	55
4	Rocky	31	47
5	Sebastian	27	72
6	Jaqueline	25	76
7	Rahul	33	79
8	David	42	44
9	Andrew	32	92
10	Ajay	51	99
11	Teresa	47	69
12	Madonna	38	73

```
1 #Trường hợp 1:  
2 #Sắp xếp dữ liệu Dataframe theo cột Score  
3 #Mặc định là sắp xếp tăng dần  
4 df.sort_values(by='Name')
```

	Name	Age	Score
10	Ajay	51	99
0	Alisa	26	89
9	Andrew	32	92
1	Bobby	27	87
2	Cathrine	25	67
8	David	42	44
6	Jaqueline	25	76
3	Madonna	24	55
12	Madonna	38	73

01

3. Sắp xếp dữ liệu

- Ví dụ:

	Name	Age	Score
0	Alisa	26	89
1	Bobby	27	87
2	Cathrine	25	67
3	Madonna	24	55
4	Rocky	31	47
5	Sebastian	27	72
6	Jaqueline	25	76
7	Rahul	33	79
8	David	42	44
9	Andrew	32	92
10	Ajay	51	99
11	Teresa	47	69
12	Madonna	38	73

```
1 #Trường hợp 2:  
2 #Sắp xếp dữ liệu Dataframe theo cột Score  
3 #Giá trị giảm dần  
4 df.sort_values(by='Score',ascending=False)
```

	Name	Age	Score
10	Ajay	51	99
9	Andrew	32	92
0	Alisa	26	89
1	Bobby	27	87
7	Rahul	33	79
6	Jaqueline	25	76
12	Madonna	38	73
5	Sebastian	27	72

02

3. Sắp xếp dữ liệu

- Trường hợp sắp xếp nhiều cột, sẽ thực hiện sắp xếp theo thứ tự các cột từ trái sang phải:

	Name	Age	Score
0	Alisa	26	89
1	Bobby	27	87
2	Cathrine	25	67
3	Madonna	24	55
4	Rocky	31	47
5	Sebastian	27	72
6	Jaqluine	25	76
7	Rahul	33	79
8	David	42	44
9	Andrew	32	92
10	Ajay	51	99
11	Teresa	47	69
12	Madonna	38	73

```
1 #Trường hợp 3:  
2 #Sắp xếp dữ liệu Dataframe theo cột Name, Score  
3 #Giá trị tăng dần  
4 df.sort_values(by=['Name', 'Score'])
```

	Name	Age	Score
10	Ajay	51	99
0	Alisa	26	89
9	Andrew	32	92
1	Bobby	27	87
2	Cathrine	25	67
8	David	42	44
6	Jaqluine	25	76
3	Madonna	24	55
12	Madonna	38	73

03

3. Sắp xếp dữ liệu

- Ví dụ:

	Name	Age	Score
0	Alisa	26	89
1	Bobby	27	87
2	Cathrine	25	67
3	Madonna	24	55
4	Rocky	31	47
5	Sebastian	27	72
6	Jaquiline	25	76
7	Rahul	33	79
8	David	42	44
9	Andrew	32	92
10	Ajay	51	99
11	Teresa	47	69
12	Madonna	38	73

```
1 #Trường hợp 4:  
2 #Sắp xếp dữ liệu Dataframe theo cột Name, Score  
3 #Giá trị cột Name tăng dần  
4 #Giá trị cột Score giảm dần  
5 df.sort_values(by=['Name','Score'],  
6               ascending=[True,False])
```

	Name	Age	Score
10	Ajay	51	99
0	Alisa	26	89
9	Andrew	32	92
1	Bobby	27	87
2	Cathrine	25	67
8	David	42	44
6	Jaquiline	25	76
12	Madonna	38	73
3	Madonna	24	55
7	Rahul	33	79
4	Rocky	31	47

04

3. Nhóm dữ liệu (groupby)

Groupby values

- `df.groupby()`: Gom nhóm các giá trị trong một DataFrame bởi các cột được chỉ định.
- Kết hợp với `sum()`, `mean()`, `max()`, `min()` để xác định các thông số theo từng nhóm.



ID	Value
1	50.30
1	123.30
1	132.90
2	50.30
2	123.30
2	132.90
2	88.90
3	50.30
3	123.30

ID	Value
1	306.50
2	395.40
3	173.60

Groupby values

- Sử dụng phương thức groupby():

	Name	Exam	Subject	Score
0	Alisa	Semester 1	Mathematics	62
1	Bobby	Semester 1	Mathematics	47
2	Cathrine	Semester 1	Mathematics	55
3	Alisa	Semester 1	Science	74
4	Bobby	Semester 1	Science	31
5	Cathrine	Semester 1	Science	77
6	Alisa	Semester 2	Mathematics	85
7	Bobby	Semester 2	Mathematics	63
8	Cathrine	Semester 2	Mathematics	42
9	Alisa	Semester 2	Science	67
10	Bobby	Semester 2	Science	89
11	Cathrine	Semester 2	Science	81

#GROUPBY CHO 1 CỘT:

#Trường hợp 1:

#Nhóm theo Học sinh (Name) và tính tổng điểm (Score)

```
result = data.groupby('Name')['Score'].sum()
result
```

```
Name
Alisa      288
Bobby      230
Cathrine    255
Name: Score, dtype: int64
```

#Có thể sử dụng reset_index() để đưa cột groupby về lại thành cột thường.

```
result = data.groupby('Name')['Score'].sum().reset_index()
result
```

	Name	Score
0	Alisa	288
1	Bobby	230
2	Cathrine	255

01

02

Groupby values

- Sử dụng phương thức groupby():

```
#Trường hợp 2:  
# Sử dụng .agg() để áp dụng nhiều phép toán khác nhau cùng lúc  
#Tính tổng điểm, điểm trung bình, điểm max, min của từng sinh viên  
result = data.groupby('Name').agg(  
    Total_Score=('Score', 'sum'),  
    Average_Score=('Score', 'mean'),  
    Max_Score=('Score', 'max'),  
    Min_Score=('Score', 'min')  
)  
result
```

03

	Total_Score	Average_Score	Max_Score	Min_Score
Name				
Alisa	288	72.00	85	62
Bobby	230	57.50	89	31
Cathrine	255	63.75	81	42

```
#GROUPBY CHO NHIỀU CỘT:  
#Trường hợp 3: Tính điểm trung bình theo từng học sinh và từng học kỳ:  
result = data.groupby(['Name', 'Exam'])['Score'].mean().reset_index()  
result
```

	Name	Exam	Score
0	Alisa	Semester 1	68.0
1	Alisa	Semester 2	76.0
2	Bobby	Semester 1	39.0
3	Bobby	Semester 2	76.0
4	Cathrine	Semester 1	66.0
5	Cathrine	Semester 2	61.5

04

Thực hành 1

Thực hành 1

Yêu cầu 1.1:

- Đọc dữ liệu từ file **Data_Movies.csv** vào biến kiểu dataframe: df_movies; Hiển thị thông tin tổng quan của biến và quan sát các đặc trưng thông kê của các thuộc tính

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12182 entries, 0 to 12181
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   adult                 12182 non-null  bool
1   original_title        12182 non-null  object
2   overview              12119 non-null  object
3   original_language     12182 non-null  object
4   revenue               12182 non-null  float64
5   status                12181 non-null  object
6   release_date          12180 non-null  object
7   production_companies  12182 non-null  object
8   vote_count            12182 non-null  float64
9   vote_average          12182 non-null  float64
dtypes: bool(1), float64(3), object(6)
memory usage: 868.6+ KB
```

	revenue	vote_count	vote_average
count	1.218200e+04	12182.000000	12182.000000
mean	4.108224e+07	388.224594	6.308644
std	1.191479e+08	891.546891	0.919185
min	0.000000e+00	31.000000	0.000000
25%	0.000000e+00	50.000000	5.700000
50%	0.000000e+00	99.000000	6.400000
75%	2.586303e+07	299.000000	7.000000
max	2.787965e+09	14075.000000	9.500000

	original_title	overview	original_language	status	release_date	production_companies
count	12182	12119	12182	12181	12180	12182
unique	11828	12099	48	5	6884	9106
top	Life	No overview found.	en	Released	2009-01-01	[]
freq	4	8	9598	12161	19	565

Thực hành 1

Yêu cầu 1.2:

- Xóa các cột dữ liệu: revenue, status, production_companies; và áp dụng thay đổi lên chính df_movies này..

Yêu cầu 1.3:

- A) Xóa tất cả các dòng dữ liệu có số lượng người vote (vote_count) dưới 100
- B) Chuyển đổi kiểu dữ liệu cột release_date về kiểu datetimes, sắp xếp dữ liệu theo thứ tự giảm dần của thời gian (bộ film mới ra mắt ở trên cùng của dữ liệu)

Thực hành 1

Yêu cầu 1.4:

- A) Hiển thị danh sách 10 bộ film có số lượng người đánh giá cao nhất (vote_cout).
- B) Hiển thị danh sách 10 bộ film có điểm đánh giá trung bình cao nhất (vote_average)

Yêu cầu 1.5:

- A) Nhóm dữ liệu theo thuộc tính **adult**, đếm số lượng bộ phim và tổng số lượt vote:

	Count_film	Total_vote
adult		
False	11933	4589195.0
True	249	140157.0

- B) Nhóm các bộ film theo ngôn ngữ gốc (**original_language**), tính tổng số bộ film, tổng số lượt vote_count và tính trung của vote_average .

	Count_film	Total_vote	Avg_vote_avg
original_language			
ab	2	108.0	3.950000
af	1	96.0	6.900000
ar	6	413.0	7.233333
bn	4	178.0	7.925000

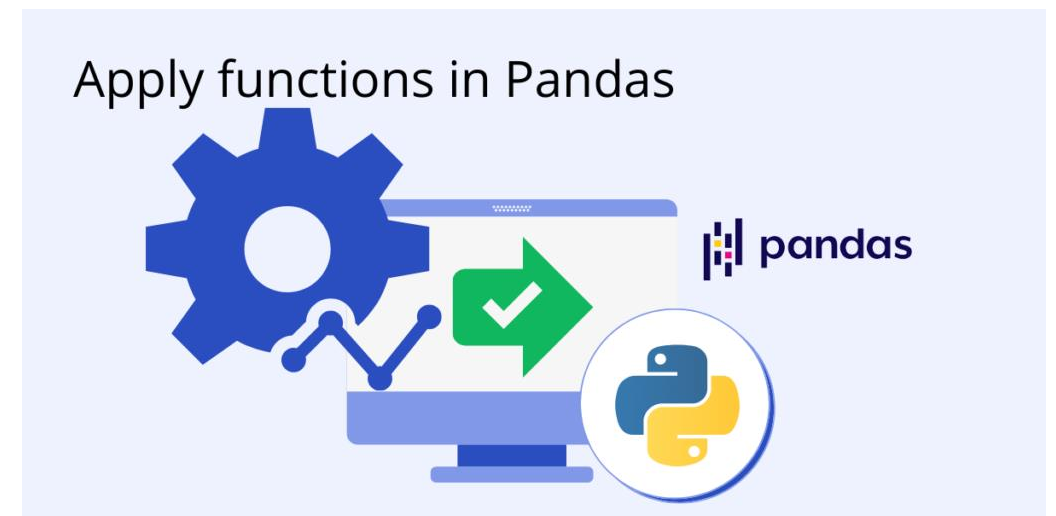
4. `apply(function)`

5 .apply(func)

- `df.apply(func)`: là một phương thức rất linh hoạt cho phép áp dụng một hàm (function) tự định nghĩa hoặc hàm sẵn có lên từng cột, từng dòng (row), hoặc từng group trong DataFrame.
- `df.apply(func)` có thể dùng với cả biến Series và DataFrame.

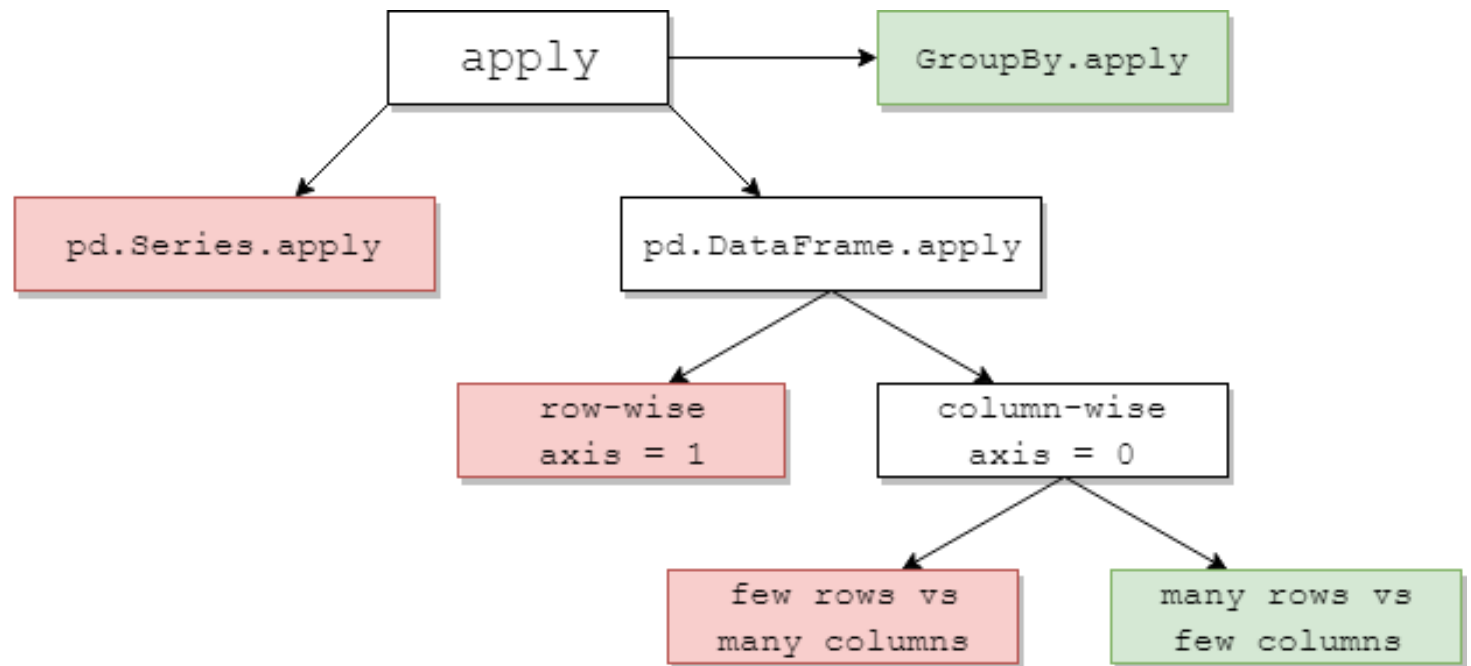
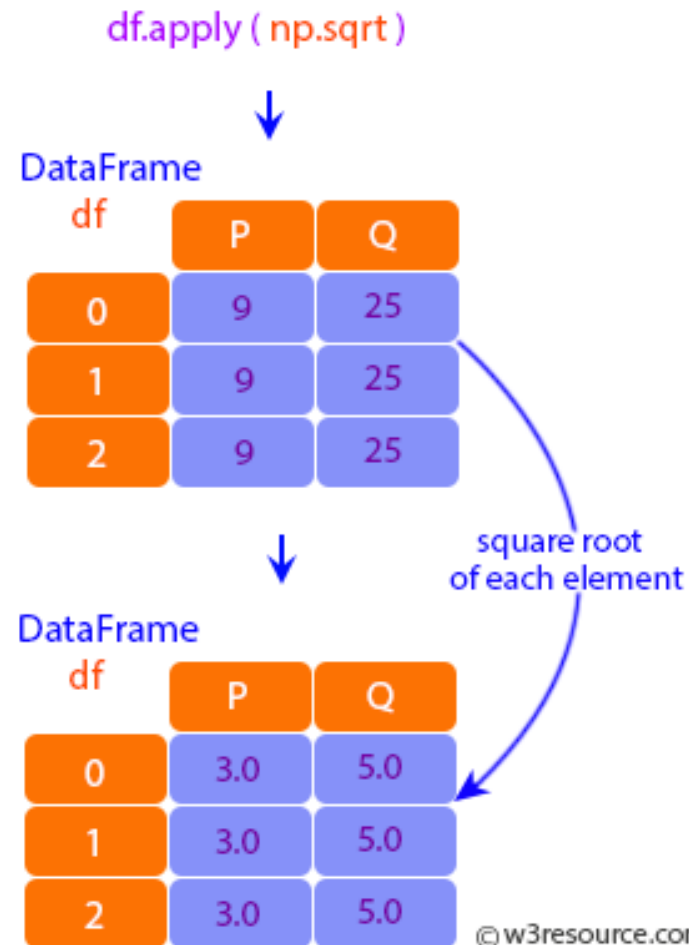
Kết hợp hiệu quả với lambda hoặc hàm tự định nghĩa cho xử lý dữ liệu tùy biến.

Sử dụng `df.apply()` sẽ làm cho code Pandas trở nên linh hoạt, đáp ứng các nhu cầu xử lý dữ liệu đa dạng trong thực tế.



5 .apply(func)

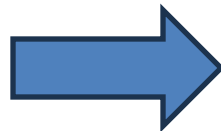
- `df.apply(func)`: Thực hiện thao tác func áp dụng cho từng cột riêng lẻ trong DataFrame, hoặc cho nhiều cột



5 .apply(func)

- Áp dụng hàm cho các phần tử trong một cột dữ liệu của DataFrame: Viết hoa các giá trị trong cột Name (3 Cách thực hiện)

	Name	Score_Math	Score_Science
0	william	86	89
1	MaSON	57	87
2	ella	75	67
3	jackson	64	55
4	lincoln	71	47
5	aubrey	67	72
6	Hudson	85	76
7	christian	33	79
8	Sawyer	42	44
9	SILAS	90	92
10	Bennett	51	93
11	kingston	47	69



```
#Trường hợp 1: Sử dụng .apply() áp dụng biến đổi giá trị cho một cột  
#Thực hiện chuẩn hóa Cột Name về dạng chung viết hoa ký tự đầu tiên  
df_student['Name_clean'] = df['Name'].apply(lambda x: x.title())  
df_student
```

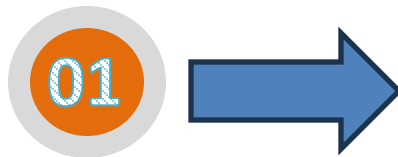
	Name	Score_Math	Score_Science	Name_clean
0	william	86	89	William
1	MaSON	57	87	Mason
2	ella	75	67	Ella
3	jackson	64	55	Jackson
4	lincoln	71	47	Lincoln
5	aubrey	67	72	Aubrey
6	Hudson	85	76	Hudson
7	christian	33	79	Christian
8	Sawyer	42	44	Sawyer
9	SILAS	90	92	Silas
10	Bennett	51	93	Bennett
11	kingston	47	69	Kingston

01

5 .apply(func)

- Áp dụng .apply() cho từng dòng/từng cột dữ liệu của DataFrame:
 - **df.apply(func, axis=1)** # Áp dụng func cho từng dòng

```
#Điểm trung bình = (score_math*2 + score_science)/3  
#Viết hàm tính điểm trung bình của sinh viên:  
def mean_point(point1,point2):  
    return round((point1*2+point2)/3,1)
```



```
#Tạo một cột Point tính điểm của từng học sinh theo hàm đã xây dựng:  
df_student['Point'] = df_student.apply(lambda row: mean_point(row['Score_Math'],  
                                                             row['Score_Science']),  
                                     axis=1)  
  
df_student
```

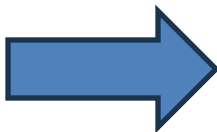
	Name	Score_Math	Score_Science	Name_clean	Point
0	william	86	89	William	87.0
1	MaSON	57	87	Mason	67.0
2	ella	75	67	Ella	72.3
3	jackson	64	55	Jackson	61.0
4	lincoln	71	47	Lincoln	63.0
5	aubrey	67	72	Aubrey	68.7
6	Hudson	85	76	Hudson	82.0
7	christian	33	79	Christian	48.3
8	Sawyer	42	44	Sawyer	42.7
9	SILAS	90	92	Silas	90.7
10	Bennett	51	93	Bennett	65.0
11	kingston	47	69	Kingston	54.3

5 .apply(func)

```
#Phân loại học sinh dựa trên điểm TB:
def classify(score):
    if score >= 85:
        return 'Giỏi'
    elif score >= 70:
        return 'Khá'
    elif score >= 50:
        return 'Trung bình'
    else:
        return 'Yếu'

#-----
#Áp dụng .apply() cho toàn bộ các dòng trong Dataframe:
df_student['Type'] = df_student.apply(lambda row: classify(row['Point']),
                                     axis=1)

df_student
```



	Name	Score_Math	Score_Science	Name_clean	Point	Type
0	william	86	89	William	87.0	Giỏi
1	MaSON	57	87	Mason	67.0	Trung bình
2	ella	75	67	Ella	72.3	Khá
3	jackson	64	55	Jackson	61.0	Trung bình
4	lincoln	71	47	Lincoln	63.0	Trung bình
5	aubrey	67	72	Aubrey	68.7	Trung bình
6	Hudson	85	76	Hudson	82.0	Khá
7	christian	33	79	Christian	48.3	Yếu
8	Sawyer	42	44	Sawyer	42.7	Yếu
9	SILAS	90	92	Silas	90.7	Giỏi
10	Bennett	51	93	Bennett	65.0	Trung bình
11	kingston	47	69	Kingston	54.3	Trung bình

5 .apply(func)

- Áp dụng .apply() cho từng cột dữ liệu của DataFrame:
 - **df.apply(func, axis=0)** # Áp dụng func cho từng cột

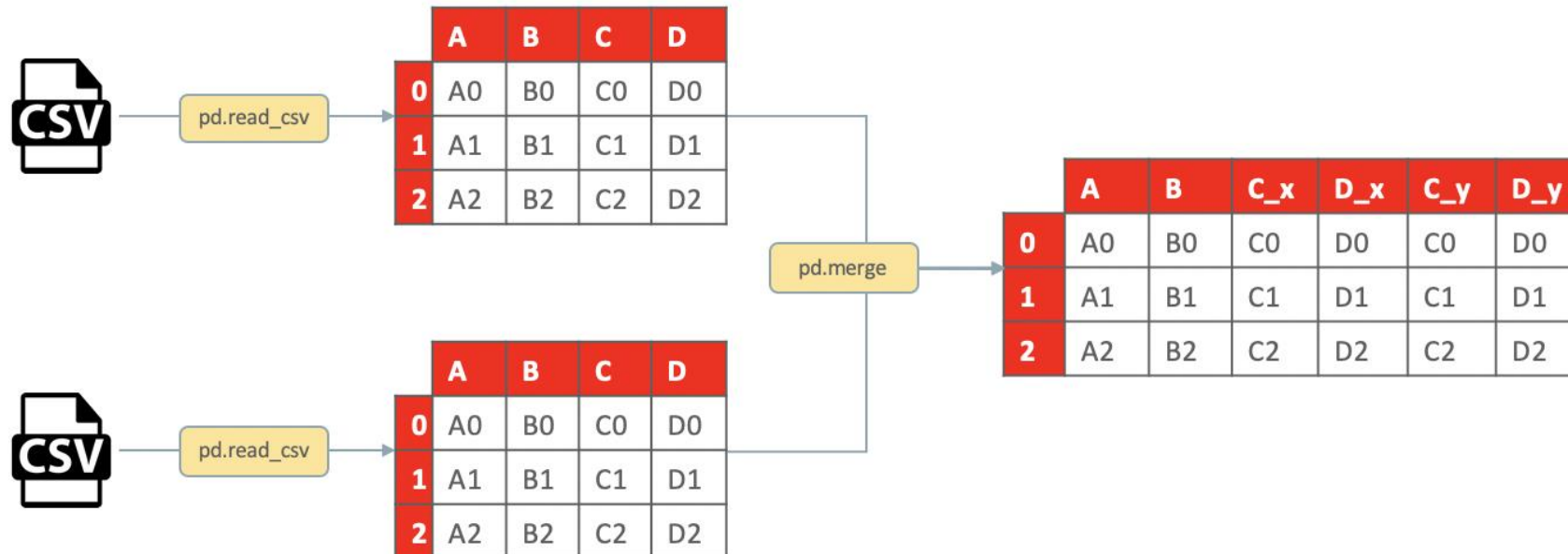
```
#Trường hợp 3: Áp dụng .apply() cho từng cột (axis=0)
#Tính giá trị min, max, mean cho các cột điểm số:
result = df_student[['Score_Math', 'Score_Science', 'Point']].apply([np.min, np.max, np.mean],
                                                                    axis=0)
result
```

	Score_Math	Score_Science	Point
min	33.0	44.0	42.700000
max	90.0	93.0	90.700000
mean	64.0	72.5	66.833333

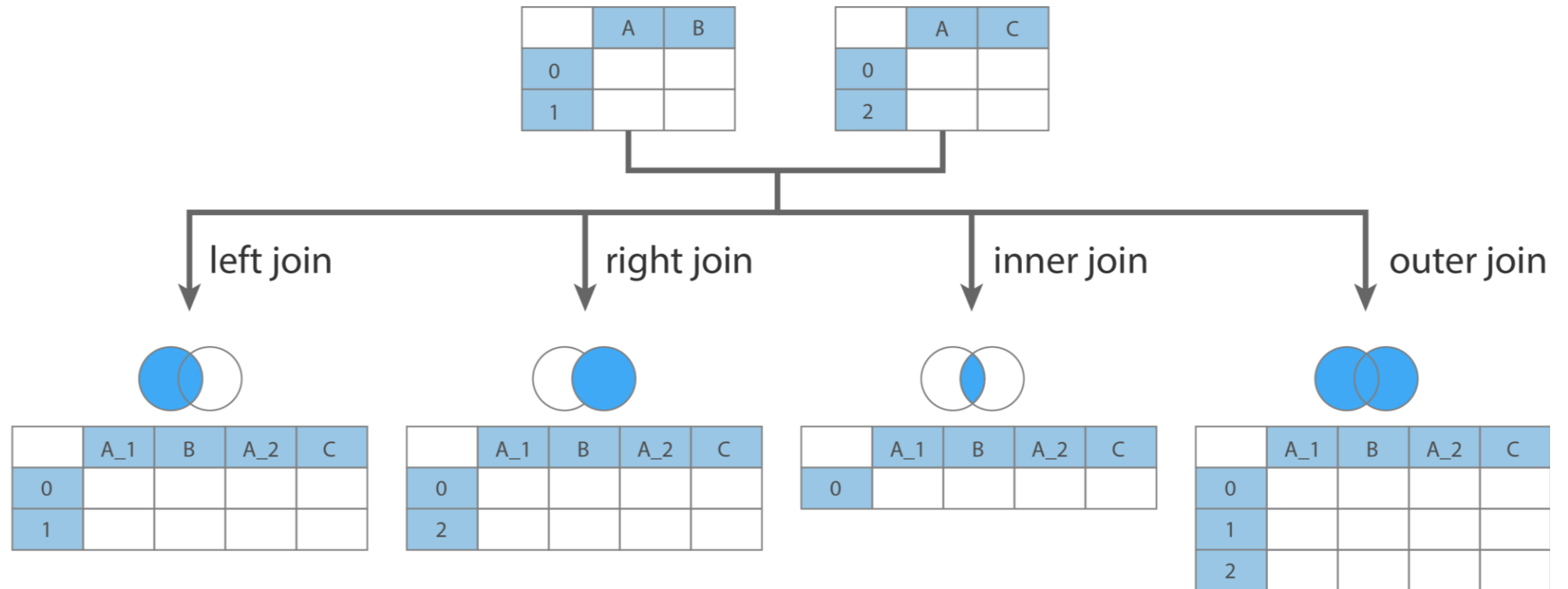
6. Trộn các DataFrame (Merge)

Trộn các DataFrame

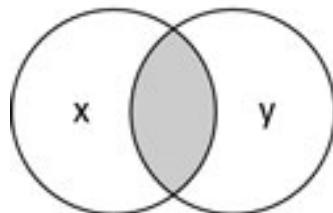
- `pd.merge(left_df, right_df, on='key', how='left' | 'right' | 'inner' | 'outer')`: Thực hiện trộn 2 DataFrame lại với nhau.
 - Left_df: DataFrame 1
 - Right_df: DataFrame2
 - On: Tên cột dùng để nối dữ liệu giữa 2 DataFrame (tên cột phải có ở trong cả 2 DataFrame 1, 2)
 - How: Cách thức trộn dữ liệu [left, right, outer, inner (default)]



Trộn các DataFrame

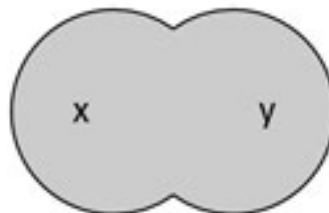


how='inner'



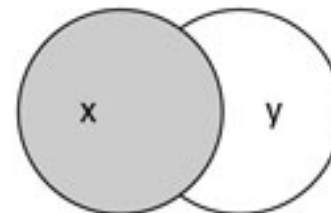
natural join

how='outer'



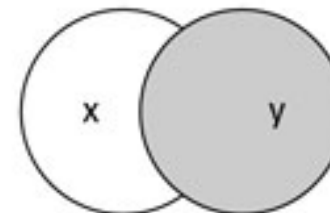
full outer join

how='left'



left outer join

how='right'



right outer join

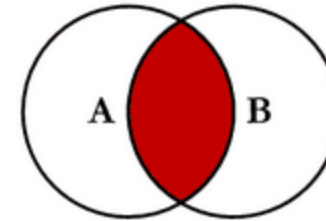
Trộn các DataFrame (inner)

	Customer_id	Product
0	1	Oven
1	2	Oven
2	3	Oven
3	4	Television
4	5	Television
5	6	Television

df1

	Customer_id	State
0	2	California
1	4	California
2	6	Texas
3	7	New York
4	8	Indiana

df2



Inner Join

```
1 #Trường hợp 1:  
2 #Inner join DataFrame  
3 inner_join_df= pd.merge(df1, df2,  
4                          on='Customer_id',  
5                          how='inner')  
6 inner_join_df
```

	Customer_id	Product	State
0	2	Oven	California
1	4	Television	California
2	6	Television	Texas

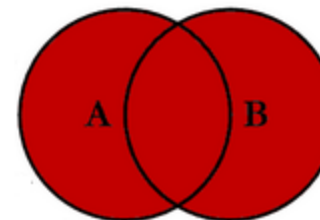
Trộn các DataFrame (outer)

	Customer_id	Product
0	1	Oven
1	2	Oven
2	3	Oven
3	4	Television
4	5	Television
5	6	Television

df1

	Customer_id	State
0	2	California
1	4	California
2	6	Texas
3	7	New York
4	8	Indiana

df2



Outer Join

```
1 #Trường hợp 2:  
2 #Outer join DataFrame  
3 inner_join_df= pd.merge(df1, df2,  
4                          on='Customer_id',  
5                          how='outer')  
6 inner_join_df
```

	Customer_id	Product	State
0	1	Oven	NaN
1	2	Oven	California
2	3	Oven	NaN
3	4	Television	California
4	5	Television	NaN
5	6	Television	Texas
6	7	NaN	New York
7	8	NaN	Indiana

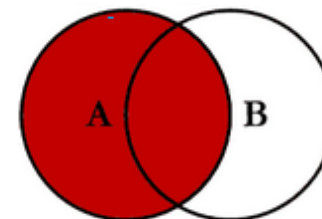
Trộn các DataFrame (left)

	Customer_id	Product
0	1	Oven
1	2	Oven
2	3	Oven
3	4	Television
4	5	Television
5	6	Television

df1

	Customer_id	State
0	2	California
1	4	California
2	6	Texas
3	7	New York
4	8	Indiana

df2



Left join

```
1 #Trường hợp 3:  
2 #Left join DataFrame  
3 inner_join_df= pd.merge(df1, df2,  
4                          on='Customer_id',  
5                          how='left')  
6 inner_join_df
```

	Customer_id	Product	State
0	1	Oven	NaN
1	2	Oven	California
2	3	Oven	NaN
3	4	Television	California
4	5	Television	NaN
5	6	Television	Texas

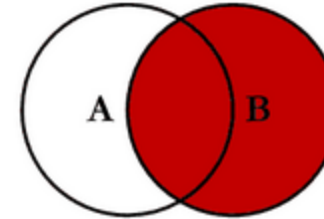
Trộn các DataFrame (right)

	Customer_id	Product
0	1	Oven
1	2	Oven
2	3	Oven
3	4	Television
4	5	Television
5	6	Television

df1

	Customer_id	State
0	2	California
1	4	California
2	6	Texas
3	7	New York
4	8	Indiana

df2



Right Join

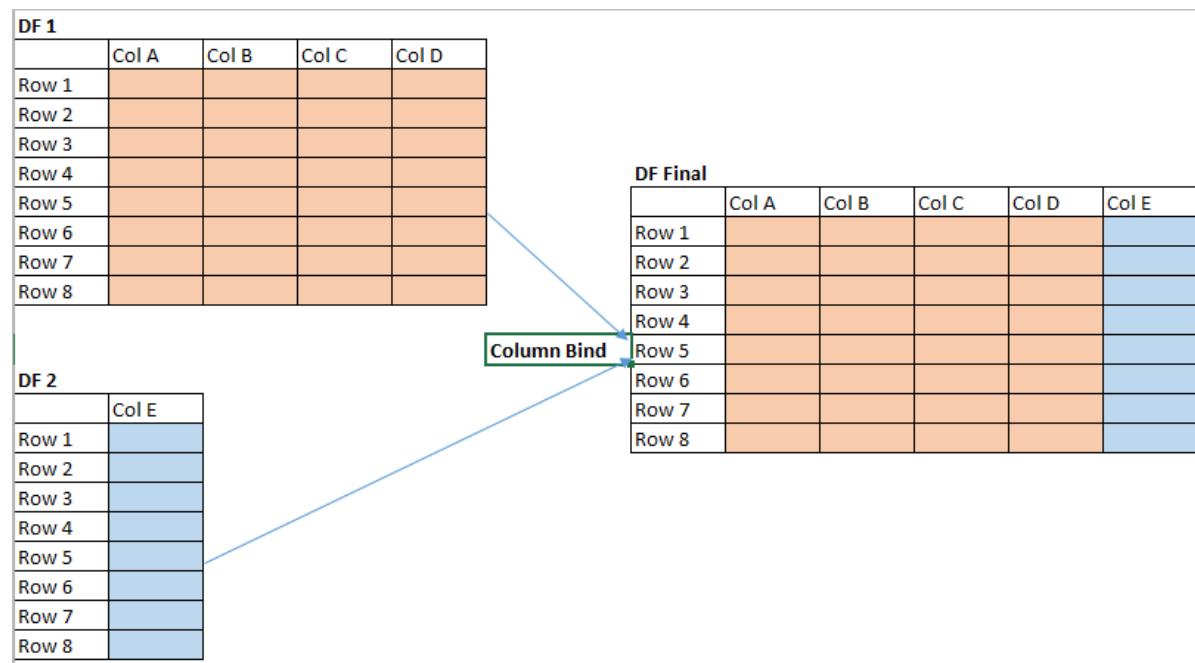
```
1 #Trường hợp 4:  
2 #Right join DataFrame  
3 inner_join_df= pd.merge(df1, df2,  
4                          on='Customer_id',  
5                          how='right')  
6 inner_join_df
```

	Customer_id	Product	State
0	2	Oven	California
1	4	Television	California
2	6	Television	Texas
3	7	NaN	New York
4	8	NaN	Indiana

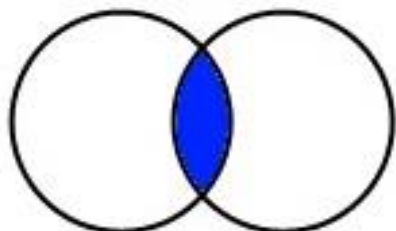
7. Nối các DataFrame (concat, append)

Nối các DataFrame theo cột

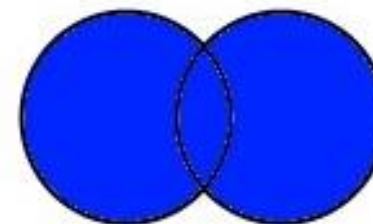
- `pd.concat([df1,df2], axis=1, join='inner'|'outer')`: Thực hiện ghép nối các hàng của DataFrame lại với (thêm cột)



INNER JOIN



FULL OUTER JOIN



Nối các DataFrame theo cột

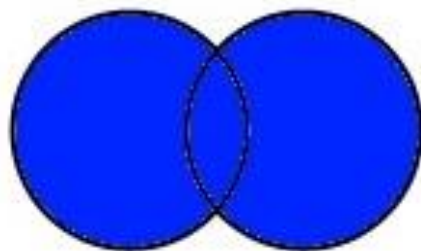
	Name	Score1	Score2
0	Alisa	62	89
1	Bobby	47	87
2	Cathrine	55	67
3	Madonna	74	55
4	Rocky	31	47
5	Sebastian	77	72
6	Jaquiline	85	76
7	Rahul	63	79
8	David	42	44

01

- `pd.concat([df1,df2], axis=1)`: sử dụng các tham số mặc định

```
1 #Trường hợp 1:  
2 #Mặc định join='outer'  
3 df_concat1 = pd.concat([df1, df2], axis=1)  
4 df_concat1
```

FULL OUTER JOIN



	Name	Score3
0	Alisa	56
1	Bobby	86
2	Cathrine	77
3	Madonna	45
4	Rocky	73
5	Sebastian	62
6	Jaquiline	74

02

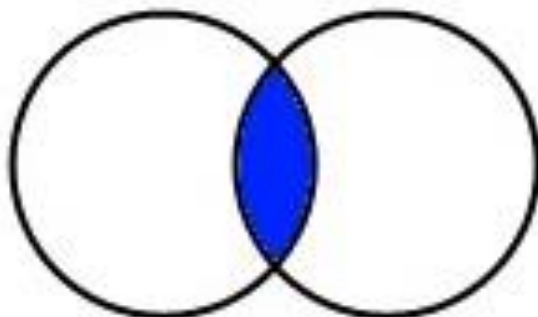
	Name	Score1	Score2	Name	Score3
0	Alisa	62	89	Alisa	56.0
1	Bobby	47	87	Bobby	86.0
2	Cathrine	55	67	Cathrine	77.0
3	Madonna	74	55	Madonna	45.0
4	Rocky	31	47	Rocky	73.0
5	Sebastian	77	72	Sebastian	62.0
6	Jaquiline	85	76	Jaquiline	74.0
7	Rahul	63	79	NaN	NaN
8	David	42	44	NaN	NaN

Nối các DataFrame theo cột

	Name	Score1	Score2
0	Alisa	62	89
1	Bobby	47	87
2	Cathrine	55	67
3	Madonna	74	55
4	Rocky	31	47
5	Sebastian	77	72
6	Jaquiline	85	76
7	Rahul	63	79
8	David	42	44

01

INNER JOIN



	Name	Score3
0	Alisa	56
1	Bobby	86
2	Cathrine	77
3	Madonna	45
4	Rocky	73
5	Sebastian	62
6	Jaquiline	74

02

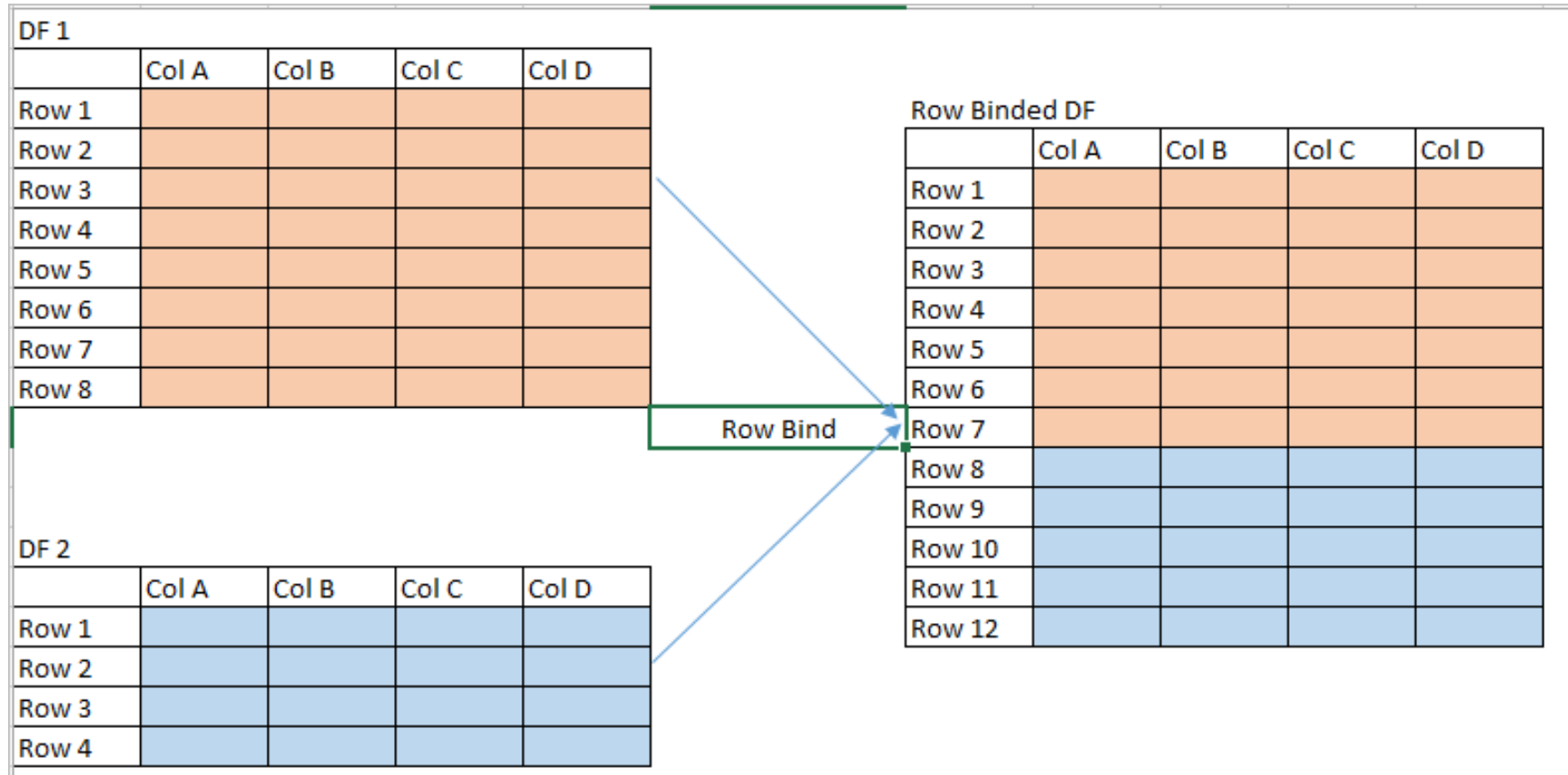
- `pd.concat([df1,df2], axis=1, join='inner')`:
Sử dụng tham số join

```
1 #Trường hợp 2:  
2 #Tham số join='inner'  
3 df_concat2 = pd.concat([df1, df2],  
4                          axis=1,  
5                          join='inner')  
6 df_concat2
```

	Name	Score1	Score2	Name	Score3
0	Alisa	62	89	Alisa	56
1	Bobby	47	87	Bobby	86
2	Cathrine	55	67	Cathrine	77
3	Madonna	74	55	Madonna	45
4	Rocky	31	47	Rocky	73
5	Sebastian	77	72	Sebastian	62
6	Jaquiline	85	76	Jaquiline	74

Nối các DataFrame theo hàng

- `pd.concat([df1,df2], axis=0, join='inner'|'outer', ignore_index=True|False)`



Nối các DataFrame theo hàng

Trường hợp các cột cùng tên:

01

	Name	Score1	Score2	Score3
0	Alisa	62	89	56
1	Bobby	47	87	86
2	Cathrine	55	67	77
3	Madonna	74	55	45
4	Rocky	31	47	73

02

	Name	Score1	Score2	Score3
0	Andrew	32	92	67
1	Ajay	71	99	97
2	Teresa	57	69	68

- `pd.concat([df1,df2]):`

```
1 #Trường hợp 1: sử dụng concat  
2 df_row = pd.concat([df1,df2])  
3 df_row
```

	Name	Score1	Score2	Score3
0	Alisa	62	89	56
1	Bobby	47	87	86
2	Cathrine	55	67	77
3	Madonna	74	55	45
4	Rocky	31	47	73
0	Andrew	32	92	67
1	Ajay	71	99	97
2	Teresa	57	69	68

Nối các DataFrame theo hàng

Trường hợp các cột khác tên:

01

	Name	Score1	Score2	Score3
0	Alisa	62	89	56
1	Bobby	47	87	86
2	Cathrine	55	67	77
3	Madonna	74	55	45
4	Rocky	31	47	73

02

	Name	Score1	Score4	Score5
0	Jack	32	72	57
1	danny	71	91	72
2	vishwa	70	89	78

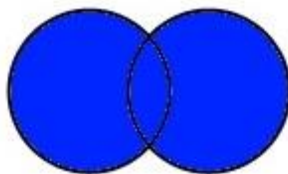
`pd.concat([df1,df3], join ='outer|inner' )`

```
1 #Trường hợp các cột khác tên
2 pd.concat([df1,df3])
```

```
1 #sử dụng tham số join='inner'
2 pd.concat([df1,df3], join ='inner')
```

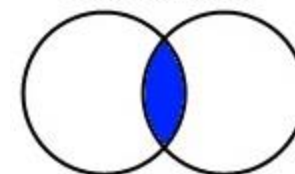
	Name	Score1	Score2	Score3	Score4	Score5
0	Alisa	62	89.0	56.0	NaN	NaN
1	Bobby	47	87.0	86.0	NaN	NaN
2	Cathrine	55	67.0	77.0	NaN	NaN
3	Madonna	74	55.0	45.0	NaN	NaN
4	Rocky	31	47.0	73.0	NaN	NaN
0	Jack	32	NaN	NaN	72.0	57.0
1	danny	71	NaN	NaN	91.0	72.0
2	vishwa	70	NaN	NaN	89.0	78.0

FULL OUTER JOIN



	Name	Score1
0	Alisa	62
1	Bobby	47
2	Cathrine	55
3	Madonna	74
4	Rocky	31
0	Jack	32
1	danny	71
2	vishwa	70

INNER JOIN



Thực hành 2

Thực hành 2

Yêu cầu 2.1:

- Đọc dữ liệu từ file **Data_Point.xlsx** vào biến kiểu dataframe:
 - df_lop1 dữ liệu điểm sheet 0 (4080130_01)
 - df_lop2 dữ liệu điểm sheet 1 (4080130_02)
 - df_lop3 dữ liệu điểm sheet 2 (4080130_03)

1	Code	A	B1	B2	C1	C2
2	1621050322	8	0	5	7.5	8
3	1621050512	6	3	7.5	8.5	9
4	1621050211	6.7	4	6.5	3	5
5	1621050827	8	6.5	8	10	9
6	1621050298	7	5	8	8.5	9
7	1621050351	4.3	5	5	6	6
8	1621050422	7	6.5	9	10	10
9	1621050281	5.3	3.5	6	8.5	8
10	1621050753	6	5	6.5	10	10
11	1621050283	6	5.5	7	8.5	8
12	1621050122	5.3	2	6	8.5	8
13	1621050203	6	8	8	10	10
14	1621050090	6	5	6.5	10	8
15	1621050802	7	0	8	0	5
16	1621050434	6	9	7	10	9.5
17	1621050240	6	9	7	10	9.5

4080130_01

4080130_02

4080130_03

Code

+

Thực hành 2

Yêu cầu 2.2:

- Nối 3 DataFrame df_lop1, df_lop2, df_lop3 thành một DataFrame df_full chứa tất cả danh sách bảng điểm của 3 lớp

```
1 df_full.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144 entries, 0 to 143
Data columns (total 6 columns):
#   Column  Non-Null Count  Dtype  
---  -
0   Code    144 non-null     int64  
1   A        144 non-null     float64
2   B1       144 non-null     float64
3   B2       144 non-null     float64
4   C1       144 non-null     float64
5   C2       144 non-null     float64
dtypes: float64(5), int64(1)
memory usage: 6.9 KB
```

	Code	A	B1	B2	C1	C2
0	1621050322	8.0	0.0	5.0	7.5	8.0
1	1621050512	6.0	3.0	7.5	8.5	9.0
2	1621050211	6.7	4.0	6.5	3.0	5.0
3	1621050827	8.0	6.5	8.0	10.0	9.0
4	1621050298	7.0	5.0	8.0	8.5	9.0
...	...	...	...	...	...	...
139	1721050290	7.0	8.0	8.0	10.0	9.0
140	1621050162	6.3	7.0	8.5	10.0	9.0
141	1721050199	6.3	0.0	7.5	10.0	6.0
142	1621050308	0.0	5.0	0.0	10.0	9.0
143	1621050034	8.0	8.0	7.5	8.5	8.0

144 rows × 6 columns

Thực hành 2

Yêu cầu 2.3:

- Trong df_full: Tạo một cột **Diem_he10** được tính dựa vào các cột tương ứng của từng hàng dữ liệu, theo công thức sau:

$$\text{Diem_he10} = 0.6 * A + 0.3 * ((B1+B2)/2) + 0.1 * ((C1+C2)/2)$$

- Làm tròn đến 1 số sau dấu phẩy

	Code	A	B1	B2	C1	C2	Diem_he10
0	1621050322	8.0	0.0	5.0	7.5	8.0	6.3
1	1621050512	6.0	3.0	7.5	8.5	9.0	6.0
2	1621050211	6.7	4.0	6.5	3.0	5.0	6.0
3	1621050827	8.0	6.5	8.0	10.0	9.0	7.9
4	1621050298	7.0	5.0	8.0	8.5	9.0	7.0
...	...	...	...	...	...	...	...
139	1721050290	7.0	8.0	8.0	10.0	9.0	7.6
140	1621050162	6.3	7.0	8.5	10.0	9.0	7.1
141	1721050199	6.3	0.0	7.5	10.0	6.0	5.7
142	1621050308	0.0	5.0	0.0	10.0	9.0	1.7
143	1621050034	8.0	8.0	7.5	8.5	8.0	8.0

144 rows × 7 columns

Thực hành 2

Yêu cầu 2.4:

- Từ cột Diem_he10 trong df_full tạo một cột Diem_chu, Diem_so theo quy đổi dưới đây:

Điểm theo thang 10	Điểm theo hệ 4	
	Điểm chữ	Điểm số
Từ 9,0 đến 10,0	A ⁺	4,0
Từ 8,5 đến cận 9,0	A	3,7
Từ 8,0 đến cận 8,4	B ⁺	3,5
Từ 7,0 đến cận 7,9	B	3,0
Từ 6,5 đến cận 7,0	C ⁺	2,5
Từ 5,5 đến cận 6,5	C	2,0
Từ 5,0 đến cận 5,5	D ⁺	1,5
Từ 4,0 đến cận 5,0	D	1,0
Từ 0,0 đến cận 4,0	F	0

	Code	A	B1	B2	C1	C2	Diem_he10	Diem_chu	Diem_so
0	1621050322	8.0	0.0	5.0	7.5	8.0	6.3	C	2.0
1	1621050512	6.0	3.0	7.5	8.5	9.0	6.0	C	2.0
2	1621050211	6.7	4.0	6.5	3.0	5.0	6.0	C	2.0
3	1621050827	8.0	6.5	8.0	10.0	9.0	7.9	B	3.0
4	1621050298	7.0	5.0	8.0	8.5	9.0	7.0	B	3.0
...	...	...	...	...	...	...	...	...	...
139	1721050290	7.0	8.0	8.0	10.0	9.0	7.6	B	3.0
140	1621050162	6.3	7.0	8.5	10.0	9.0	7.1	B	3.0
141	1721050199	6.3	0.0	7.5	10.0	6.0	5.7	C	2.0
142	1621050308	0.0	5.0	0.0	10.0	9.0	1.7	F	0.0
143	1621050034	8.0	8.0	7.5	8.5	8.0	8.0	B+	3.5

144 rows × 9 columns

Thực hành 2

Yêu cầu 2.5:

- Tạo một DataFrame `df_diem_ok` chỉ lấy dữ liệu các cột ['Code', 'Diem_he10', 'Diem_chu', 'Diem_so'] từ `df_full`

```
1 df_diem_ok.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144 entries, 0 to 143
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype  
---  -
0   Code        144 non-null   int64  
1   Diem_he10    144 non-null   float64
2   Diem_chu     144 non-null   object  
3   Diem_so      144 non-null   float64
dtypes: float64(2), int64(1), object(1)
memory usage: 4.6+ KB
```

	Code	Diem_he10	Diem_chu	Diem_so
0	1621050322	6.3	C	2.0
1	1621050512	6.0	C	2.0
2	1621050211	6.0	C	2.0
3	1621050827	7.9	B	3.0
4	1621050298	7.0	B	3.0
...	...	...	...	...
139	1721050290	7.6	B	3.0
140	1621050162	7.1	B	3.0
141	1721050199	5.7	C	2.0
142	1621050308	1.7	F	0.0
143	1621050034	8.0	B+	3.5

144 rows × 4 columns

Thực hành 2

Yêu cầu 2.6:

- Đọc dữ liệu trong sheet: code của file excel Data_point vào DataFrame **df_code**.
- Trộn (merge) dữ liệu của df_code và df_diem_ok để ghép phách cho bảng điểm và lưu vào DataFrame **df_finaly**.
- Thực hiện lưu dữ liệu trong DataFrame df_finaly ra file excel: **Diem_4080130.xlsx**

1	df_finaly						
	Code	Name	Birth	Class	Diem_he10	Diem_chu	Diem_so
0	1421050452	Nguyễn Duy Khánh	28/03/1995	DCCTPM59_1	0.0	F	0.0
1	1421050514	Vũ Trà My	01/01/1995	DCCTPM59_1	7.6	B	3.0
2	1521020083	Tạ Văn Được	20/08/1996	DCCTPM60_1	7.1	B	3.0
3	1521050138	Nguyễn Hữu Trang	04/10/1997	DCCTPM60_1	5.2	D+	1.5
4	1521050164	Phí Đình Thành	19/05/1997	DCCTPM60_1	0.0	F	0.0
...	...	...	...	...	...	...	...
139	1721050290	Nguyễn Hoài Thương	15/01/1999	DCCTPM62A	7.6	B	3.0
140	1721050401	Nguyễn Đức Nguyên	20/06/1999	DCCTPM62B	7.8	B	3.0
141	1721050524	Nguyễn Thị Anh	18/05/1999	DCCTPM62A	7.5	B	3.0
142	1721050707	Nguyễn Thị Lý	21/08/1994	DCCTPM62B	9.3	A+	4.0
143	1931050001	Lưu Quang Linh	17/05/1988	LCCTCT64HN	7.4	B	3.0

144 rows × 7 columns

Nâng cao chất lượng dữ liệu

7. Phát hiện và Xử lý dữ liệu trùng lặp

2. Xử lý các hàng trùng lặp



	Pet	Color	Eyes
0	Cat	Brown	Black
1	Dog	Golden	Black
2	Dog	Golden	Black
3	Dog	Golden	Brown
4	Cat	Black	Green



	Pet	Color	Eyes
0	Cat	Brown	Black
1	Dog	Golden	Black
3	Dog	Golden	Brown
4	Cat	Black	Green

Drop duplicates

Drop Duplicate Pandas

drop_duplicates()

- **df.duplicated():** để tìm kiếm các hàng trùng lặp

2. Xử lý các hàng trùng lặp

Quá trình tích hợp dữ liệu từ nhiều nguồn, có thể dẫn tới nhiều bản ghi bị trùng lặp. Việc phát hiện và xử lý các dữ liệu trùng lặp này cũng rất quan trọng để nâng cao chất lượng của dữ liệu trước khi phân tích.

	id	first_name	last_name	email
▶	1	Carine	Schmitt	carine.schmitt@verizon.net
	4	Janine	Labrune	janine.labrune@aol.com
	6	Janine	Labrune	janine.labrune@aol.com
	2	Jean	King	jean.king@me.com
	12	Jean	King	jean.king@me.com
	5	Jonas	Bergulfsen	jonas.bergulfsen@mac.com
	10	Julie	Murphy	julie.murphy@yahoo.com
	11	Kwai	Lee	kwai.lee@google.com
	3	Peter	Ferguson	peter.ferguson@google.com
	9	Roland	Keitel	roland.keitel@yahoo.com
	14	Roland	Keitel	roland.keitel@yahoo.com
	7	Susan	Nelson	susan.nelson@comcast.net
	13	Susan	Nelson	susan.nelson@comcast.net
	8	Zbyszek	Piestrzeniewicz	zbyszek.piestrzeniewicz@att.net

Để phát các bản ghi trùng lặp trong dữ liệu không khó, sử dụng bộ lọc hoặc các phương thức được cung cấp sẵn trong các gói thư viện có thể dễ dàng thống kê số lượng bản ghi trùng lặp trong dữ liệu và liệt kê chi tiết danh sách các bản ghi này.

Với các bản ghi trùng lặp, chúng ta cũng sẽ có các phương án xử lý riêng phù hợp. Một số lựa chọn để xử lý dữ liệu trùng lặp bao gồm:

- Xóa tất cả các bản ghi trùng lặp trong dữ liệu
- Giữ lại bản ghi đầu tiên, xóa tất cả các bản ghi trùng lặp phía sau
- Giữ lại bản ghi cuối cùng, xoát tất cả các bản ghi trùng lặp phía trước

2. Xử lý các hàng trùng lặp

- `df.drop_duplicates(keep='first')`: mặc định – Giữ hàng trùng lặp đầu tiên

	Name	Age	Score	
0	Alisa	26	85.0	3
1	raghu	23	31.0	2
2	jodha	23	55.0	1
3	jodha	23	55.0	
4	raghu	23	31.0	2
5	Cathrine	24	77.0	
6	Alisa	26	85.0	3
7	Bobby	24	63.0	
8	Bobby	22	42.0	
9	Alisa	26	85.0	3
10	raghu	23	31.0	2
11	Cathrine	24	NaN	



```
#Trường hợp 1:giữ lại hàng đầu tiên xóa tất cả các hàng trùng lặp phía sau  
#Sử dụng df.drop_duplicates(keep='first') – mặc định  
df1 = df.drop_duplicates(keep='first')  
display('df','df1')
```

df				df1			
	Name	Age	Score		Name	Age	Score
0	Alisa	26	85.0	0	Alisa	26	85.0
1	raghu	23	31.0	1	raghu	23	31.0
2	jodha	23	55.0	2	jodha	23	55.0
3	jodha	23	55.0	5	Cathrine	24	77.0
4	raghu	23	31.0	7	Bobby	24	63.0
5	Cathrine	24	77.0	8	Bobby	22	42.0
6	Alisa	26	85.0	11	Cathrine	24	NaN
7	Bobby	24	63.0				
8	Bobby	22	42.0				
9	Alisa	26	85.0				
10	raghu	23	31.0				
11	Cathrine	24	NaN				

2. Xử lý các hàng trùng lặp

- `df.drop_duplicates(keep='last')`: giữ hàng trùng lặp cuối cùng

	Name	Age	Score	
0	Alisa	26	85.0	3
1	raghu	23	31.0	2
2	jodha	23	55.0	1
3	jodha	23	55.0	
4	raghu	23	31.0	2
5	Cathrine	24	77.0	
6	Alisa	26	85.0	3
7	Bobby	24	63.0	
8	Bobby	22	42.0	
9	Alisa	26	85.0	3
10	raghu	23	31.0	2
11	Cathrine	24	NaN	



```
#Trường hợp 2: Giữ lại các hàng trùng lặp cuối cùng,  
# xóa tất cả các hàng trùng lặp phía trên  
#Sử dụng df.drop_duplicates(keep='last')  
df2=df.drop_duplicates(keep='last')  
  
display('df', 'df2')
```

df				df2			
	Name	Age	Score		Name	Age	Score
0	Alisa	26	85.0	3	jodha	23	55.0
1	raghu	23	31.0	5	Cathrine	24	77.0
2	jodha	23	55.0	7	Bobby	24	63.0
3	jodha	23	55.0	8	Bobby	22	42.0
4	raghu	23	31.0	9	Alisa	26	85.0
5	Cathrine	24	77.0	10	raghu	23	31.0
6	Alisa	26	85.0	11	Cathrine	24	NaN
7	Bobby	24	63.0				
8	Bobby	22	42.0				
9	Alisa	26	85.0				
10	raghu	23	31.0				
11	Cathrine	24	NaN				

2. Xử lý các hàng trùng lặp

- `df.drop_duplicates(keep=False)`: Xóa tất cả các hàng trùng lặp

	Name	Age	Score	
0	Alisa	26	85.0	3
1	raghu	23	31.0	2
2	jodha	23	55.0	1
3	jodha	23	55.0	
4	raghu	23	31.0	2
5	Cathrine	24	77.0	
6	Alisa	26	85.0	3
7	Bobby	24	63.0	
8	Bobby	22	42.0	
9	Alisa	26	85.0	3
10	raghu	23	31.0	2
11	Cathrine	24	NaN	



```
#Trường hợp 3: Xóa hết các hàng trùng lặp khỏi DataFrame  
#Sử dụng df.drop_duplicates(keep=False)  
df3=df.drop_duplicates(keep=False)  
  
display('df', 'df3')
```

df

	Name	Age	Score
0	Alisa	26	85.0
1	raghu	23	31.0
2	jodha	23	55.0
3	jodha	23	55.0
4	raghu	23	31.0
5	Cathrine	24	77.0
6	Alisa	26	85.0
7	Bobby	24	63.0
8	Bobby	22	42.0
9	Alisa	26	85.0
10	raghu	23	31.0
11	Cathrine	24	NaN

df3

	Name	Age	Score
5	Cathrine	24	77.0
7	Bobby	24	63.0
8	Bobby	22	42.0
11	Cathrine	24	NaN

2. Xử lý các hàng trùng lặp

- `df.drop_duplicates([name columns], keep='first' | 'last' | False):`
- Xóa các hàng dữ liệu trùng lặp nhau ở các cột được chỉ định

	Name	Age	Score
0	Alisa	26	85.0
1	raghu	23	31.0
2	jodha	23	55.0
3	jodha	23	55.0
4	raghu	23	31.0
5	Cathrine	24	77.0
6	Alisa	26	85.0
7	Bobby	24	63.0
8	Bobby	22	42.0
9	Alisa	26	85.0
10	raghu	23	31.0
11	Cathrine	24	NaN

01

```
1 #Trường hợp 4:
2 #Sử dụng df.drop_duplicates()
3 #Loại bỏ các hàng trùng nhau theo cột Name
4 df4=df.drop_duplicates(['Name'],keep='first')
5 df4
```

	Name	Age	Score
0	Alisa	26	85.0
1	raghu	23	31.0
2	jodha	23	55.0
5	Cathrine	24	77.0
7	Bobby	24	63.0

```
1 #Trường hợp 5:
2 #Sử dụng df.drop_duplicates()
3 #Loại bỏ các hàng trùng nhau theo cột Name, Age
4 df5=df.drop_duplicates(['Name','Age'],
5                         keep='first')
6 df5
```

	Name	Age	Score
0	Alisa	26	85.0
1	raghu	23	31.0
2	jodha	23	55.0
5	Cathrine	24	77.0
7	Bobby	24	63.0
8	Bobby	22	42.0

02

8. Phát hiện và xử lý giá trị khuyết thiếu

Phát hiện và xử lý missing data

Không phải tất cả các bộ dữ liệu đều đầy đủ, giá trị khuyết thiếu (missing values) là vấn đề rất thường gặp trong dữ liệu, việc phát hiện và xử lý chúng là yêu cầu quan trọng trong quá trình làm sạch dữ liệu

Complete data	
Age	IQ score
25	133
26	121
29	91
30	105
30	110
31	98
44	118
46	93
48	141
51	104
51	116
54	97

MAR	
Age	IQ score
25	
26	
29	
30	
30	
31	
44	118
46	93
48	141
51	104
51	116
54	97

MCAR	
Age	IQ score
25	
26	121
29	91
30	
30	110
31	
44	118
46	93
48	
51	
51	116
54	

MNAR	
Age	IQ score
25	133
26	121
29	
30	
30	110
31	
44	118
46	
48	141
51	
51	116
54	

- **Dữ liệu thiếu ngẫu nhiên (Missing at Random – MAR):** Sự mất mát dữ liệu ở đây là ngẫu nhiên, tuy nhiên vẫn có mối quan hệ hệ thống giữa dữ liệu bị mất và dữ liệu được quan sát.
- **Dữ liệu thiếu hoàn toàn ngẫu nhiên (Missing Completely at Random – MCAR):** Sự mất mát dữ liệu tại các điểm quan sát là hoàn toàn ngẫu nhiên, và không có bất kỳ một mối quan hệ hay sự liên hệ nào giữa dữ liệu thiếu với các dữ liệu quan sát khác.
- **Dữ liệu thiếu không ngẫu nhiên (Missing Not at Random – MNAR):** Sự mất mát dữ liệu không phải là ngẫu nhiên mà có một mối quan hệ xu hướng giữa giá trị bị thiếu và giá trị không bị thiếu trong một biến.

Phát hiện và xử lý missing data

Các giá trị dữ liệu khuyết thiếu xuất hiện dưới nhiều dạng khác nhau, phổ biến nhất là dữ liệu trống (rỗng), hoặc có thể được ký hiệu là NA, N/A, -1, 99, 999... Do đó, để có thể phát hiện ra dữ liệu khuyết thiếu trong tập dữ liệu cần phải hiểu rõ dữ liệu đang xử lý, các thuộc tính và kiểu dữ liệu thiếu được biểu diễn ứng với từng thuộc tính

	A	B	C	D	E	F	G
1	time	Ha Noi	Vinh	Da Nang	Nha Trang	Ho Chi Minh	Ca Mau
2	00 15-9-2019	25.65	24.79	24.01	25.06	25.48	24.97
3	01 15-9-2019		24.21	24.02	24.93	25.16	24.83
4	02 15-9-2019	25.05	23.73	23.89	24.79	24.8	24.55
5	03 15-9-2019	24.79	23.36	23.83		24.74	24.48
6	04 15-9-2019	24.59	23.05	23.69	24.82	24.8	24.38
7	05 15-9-2019	24.4		23.52	24.79	24.87	24.4
8	06 15-9-2019	24.38	22.79	23.68	25.1	24.71	24.41
9	07 15-9-2019	26.72	25.61	24.92	26.56	25.03	24.91
10	08 15-9-2019	28.84	26.93	26.51	26.53	25.75	25.85
11	09 15-9-2019	30.29	28.72	27.48	26.95	26.64	26.79
12	10 15-9-2019		29.97			27.68	27.53
13	11 15-9-2019	32.05	28.93	26.86	27.38	28.43	28.98
14	12 15-9-2019	31.31	28.94	26.65	27.47	28.29	29.24
15	13 15-9-2019	30.95		27.83	27.44	28	30.66
16	14 15-9-2019	30.56	30.62	26.49	27.16	27.67	30.97
17	15 15-9-2019	31.13	30.58	26.29	26.68	27.29	30.59
18	16 15-9-2019	30.8	30.2		26.45	27.29	29.13
19	17 15-9-2019	29.94	29.36	25.8	26.67	26.69	28.72
20	18 15-9-2019	28.53	27.48	24.82	25.92	25.81	27.46
21	19 15-9-2019	28.89	27.03	24.93	25.88	25.93	27.07
22	20 15-9-2019	28.06	26.41	24.7		25.97	26.75
23	21 15-9-2019	27.43	26.2	24.41	25.62	25.94	26.32
24	22 15-9-2019	26.98	25.79	24.17	25.6	25.9	26.29
25	23 15-9-2019	26.68	25.31	23.81	25.53	25.8	26.36
26							
27							

2P.las

~A	DEPTH	GR	NPHI	RHOB	RT
500.	1000	64.5768	-999.0000	-999.0000	1.3877
500.	2524	63.8732	-999.0000	-999.0000	1.1355
500.	4048	63.1720	-999.0000	-999.0000	0.8908
500.	5572	62.5449	-999.0000	-999.0000	0.8785
500.	7096	61.8662	-999.0000	-999.0000	0.8731
500.	8620	60.4208	-999.0000	-999.0000	0.9706
501.	0144	59.0552	-999.0000	-999.0000	1.0608
501.	1668	58.4555	-999.0000	-999.0000	1.0807
501.	3192	57.9431	-999.0000	-999.0000	1.0818
501.	4716	58.0369	-999.0000	-999.0000	0.9526
501.	6240	58.2013	-999.0000	-999.0000	0.8494
501.	7764	58.7432	-999.0000	-999.0000	0.8856
501.	9288	59.5950	-999.0000	-999.0000	0.9073
502.	0812	61.7766	-999.0000	-999.0000	0.8668
502.	2336	63.8182	-999.0000	-999.0000	0.8575
502.	3860	65.3647	-999.0000	-999.0000	0.9586
502.	5384	66.5884	-999.0000	-999.0000	1.0321
502.	6908	66.8538	-999.0000	-999.0000	1.0236
502.	8432	67.2312	-999.0000	-999.0000	0.9932
502.	9956	67.8915	-999.0000	-999.0000	0.9071
503.	1480	68.2685	-999.0000	-999.0000	0.8446
503.	3004	68.0294	-999.0000	-999.0000	0.8334
503.	4528	67.1629	-999.0000	-999.0000	0.8970
503.	6052	65.1127	-999.0000	-999.0000	1.1015
503.	7576	63.1847	-999.0000	-999.0000	1.2438
503.	9100	61.4579	-999.0000	-999.0000	1.2833

Việc **phát hiện và thống kê dữ liệu thiếu** không khó, sử dụng các công cụ, hoặc các hàm xây dựng có thể dễ dàng phát hiện ra chúng.

a. Phát hiện missing data

- Thống kê dữ liệu missing trong dataframe:

- `df.isnull().sum()`

```
1 #Thống kê số liệu missing trong Data frame
2 #Theo từng cột
3 print('Số lượng missing data trong file dữ liệu:')
4 print(data_temp.isnull().sum())
```

Số lượng missing data trong file dữ liệu:

time	0
Ha Noi	2
Vinh	2
Da Nang	2
Nha Trang	3
Ho Chi Minh	0
Ca Mau	0

dtype: int64

- Xây dựng hàm thống kê `missing_values()`

```
1 #Xây dựng hàm thống kê dữ liệu missing trong dataframe:
2 #-----
3 #Đầu vào của hàm là 1 biến Dataframe
4 #Đầu ra bao gồm các thông số:
5 #Tổng số cột của file dữ liệu
6 #Tổng số cột có chứa dữ liệu missing
7 #Danh sách các cột chứa dữ liệu missing với 2 thống số:
8 #Tổng số giá trị missing tương ứng với cột đó
9 #Tỷ lệ % dữ liệu missing trên tổng số dữ liệu của cột
10 def missing_values(df):
11     mis_val = df.isnull().sum()
12     mis_val_percent = 100 * df.isnull().sum() / len(df)
13     mis_val_table = pd.concat([mis_val, mis_val_percent], axis=1)
14     mis_val_table_ren_columns = mis_val_table.rename(
15         columns = {0 : 'Số giá trị Missing', 1 : 'Tỷ lệ % missing'})
16     mis_val_table_ren_columns = mis_val_table_ren_columns[
17         mis_val_table_ren_columns.iloc[:,1] != 0].sort_values(
18         'Tỷ lệ % missing', ascending=False).round(1)
19     print ("File dữ liệu bao gồm có: " + str(df.shape[1]) + " cột.\n"
20           "Có " + str(mis_val_table_ren_columns.shape[0]) +
21           " cột chứa missing values.")
22     return mis_val_table_ren_columns
```

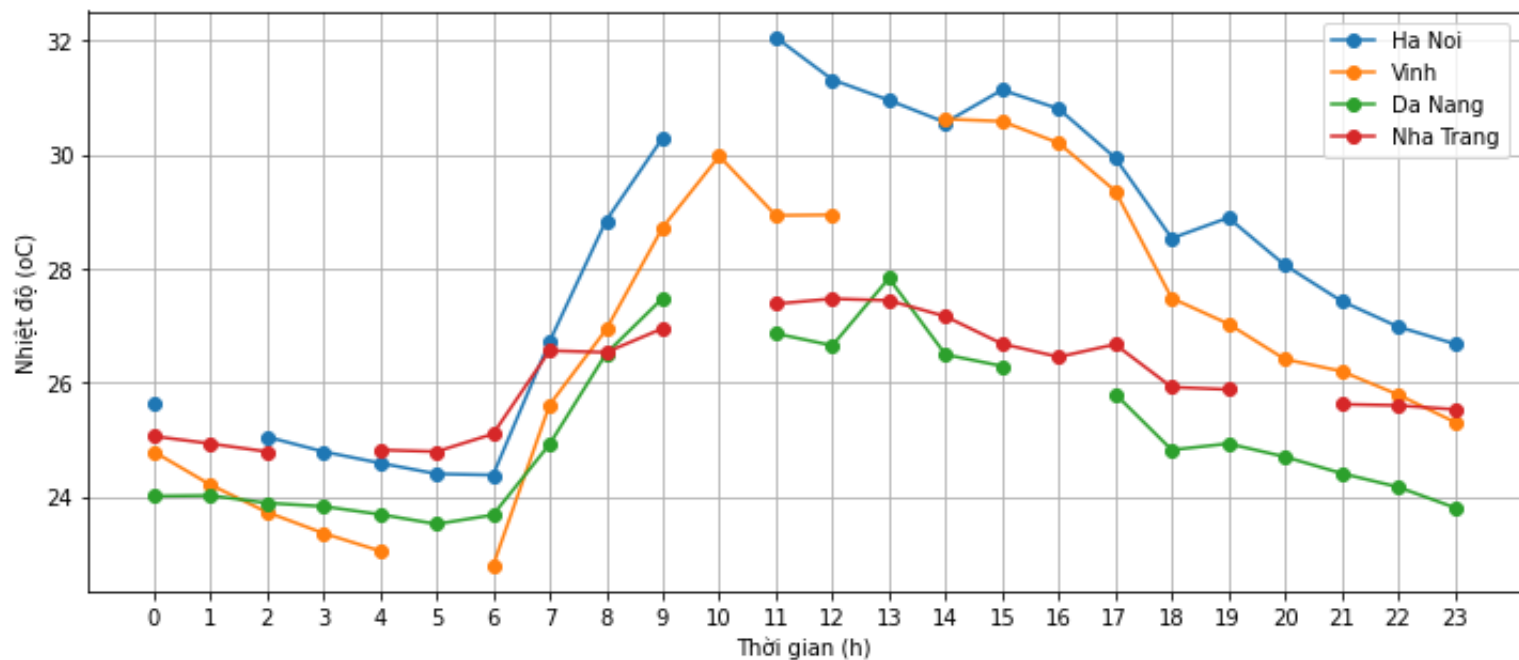
a. Phát hiện missing data

- Xây dựng hàm thống kê `missing_values()`

```
1 missing_values(data_temp)
```

File dữ liệu bao gồm có: 7 cột.
Có 4 cột chứa missing values.

	Số giá trị Missing	Tỷ lệ % missing
Nha Trang	3	12.5
Ha Noi	2	8.3
Vinh	2	8.3
Da Nang	2	8.3



a.Phát hiện missing data

– Liệt kê các hàng chứa missing trong Dataframe:

```
#Liệt kê các dòng dữ liệu missing
#1. Liệt kê toàn bộ các dòng chứa missing của Dataframe
data_temp[data_temp.isnull().any(axis=1)]
```

	time	Ha Noi	Vinh	Da Nang	Nha Trang	Ho Chi Minh	Ca Mau
1	01 15-9-2019	NaN	24.21	24.02	24.93	25.16	24.83
3	03 15-9-2019	24.79	23.36	23.83	NaN	24.74	24.48
5	05 15-9-2019	24.40	NaN	23.52	24.79	24.87	24.40
10	10 15-9-2019	NaN	29.97	NaN	NaN	27.68	27.53
13	13 15-9-2019	30.95	NaN	27.83	27.44	28.00	30.66
16	16 15-9-2019	30.80	30.20	NaN	26.45	27.29	29.13
20	20 15-9-2019	28.06	26.41	24.70	NaN	25.97	26.75

```
#2. Liệt kê theo từng cột dữ liệu chỉ định: Ha Noi
data_temp[data_temp['Ha Noi'].isnull()]
```

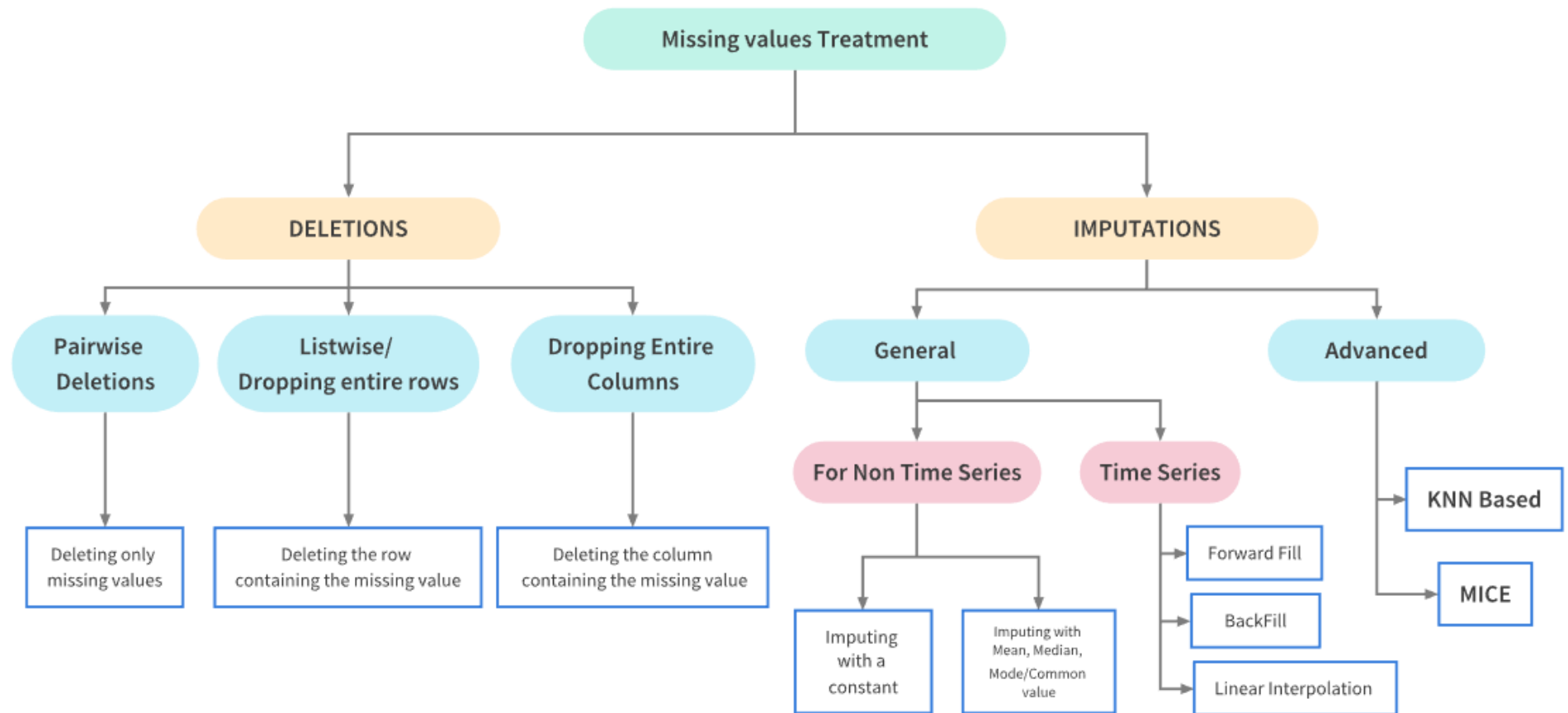
	time	Ha Noi	Vinh	Da Nang	Nha Trang	Ho Chi Minh	Ca Mau
1	01 15-9-2019	NaN	24.21	24.02	24.93	25.16	24.83
10	10 15-9-2019	NaN	29.97	NaN	NaN	27.68	27.53

```
#2. Liệt kê theo từng cột dữ liệu chỉ định: Nha Trang
data_temp[data_temp['Nha Trang'].isnull()]
```

	time	Ha Noi	Vinh	Da Nang	Nha Trang	Ho Chi Minh	Ca Mau
3	03 15-9-2019	24.79	23.36	23.83	NaN	24.74	24.48
10	10 15-9-2019	NaN	29.97	NaN	NaN	27.68	27.53
20	20 15-9-2019	28.06	26.41	24.70	NaN	25.97	26.75

b. Xử lý missing data

Sau khi phát hiện dữ liệu khuyết thiếu, cần phải xử lý chúng sao cho phù hợp. Việc lựa chọn phương pháp nào phụ thuộc vào từng thuộc tính, loại dữ liệu và bài toán cụ thể. Các phương pháp xử lý dữ liệu thiếu cơ bản được phân thành 2 nhóm chính:



b.Xử lý missing data

df.dropna(axis=0) →
loại bỏ hàng

```
1 #1) Phương pháp 1:Loại bỏ các dữ liệu missing (Deletion)
2
3 #Xóa toàn bộ các hàng chứa missing data: axis=0 -> xóa hàng
4 data_new = data_temp.dropna(axis=0,how='any')
5 #Kết quả sau khi loại bỏ các row chứa missing
6 print(data_new)
```

		time	Ha Noi	Vinh	Da Nang	Nha Trang	Ho Chi Minh	Ca Mau
0	00	15-9-2019	25.65	24.79	24.01	25.06	25.48	24.97
2	02	15-9-2019	25.05	23.73	23.89	24.79	24.80	24.55
4	04	15-9-2019	24.59	23.05	23.69	24.82	24.80	24.38
6	06	15-9-2019	24.38	22.79	23.68	25.10	24.71	24.41
7	07	15-9-2019	26.72	25.61	24.92	26.56	25.03	24.91
8	08	15-9-2019	28.84	26.93	26.51	26.53	25.75	25.85
9	09	15-9-2019	30.29	28.72	27.48	26.95	26.64	26.79
11	11	15-9-2019	32.05	28.93	26.86	27.38	28.43	28.98
12	12	15-9-2019	31.31	28.94	26.65	27.47	28.29	29.24
14	14	15-9-2019	30.56	30.62	26.49	27.16	27.67	30.97
15	15	15-9-2019	31.13	30.58	26.29	26.68	27.29	30.59
17	17	15-9-2019	29.94	29.36	25.80	26.67	26.69	28.72
18	18	15-9-2019	28.53	27.48	24.82	25.92	25.81	27.46
19	19	15-9-2019	28.89	27.03	24.93	25.88	25.93	27.07
21	21	15-9-2019	27.43	26.20	24.41	25.62	25.94	26.32
22	22	15-9-2019	26.98	25.79	24.17	25.60	25.90	26.29
23	23	15-9-2019	26.68	25.31	23.81	25.53	25.80	26.36

b.Xử lý missing data

df.dropna(axis=1) →
loại bỏ cột

```
1 #1) Phương pháp 1:Loại bỏ các dữ liệu missing (Deletion)
2
3 #Xóa toàn bộ các cột chứa missing data: axis=1 -> xóa cột
4 data_new = data_temp.dropna(axis=1,how='any')
5 #Kết quả sau khi loại bỏ các cột chứa missing
6 print(data_new)
```

	time	Ho Chi Minh	Ca Mau
0	00 15-9-2019	25.48	24.97
1	01 15-9-2019	25.16	24.83
2	02 15-9-2019	24.80	24.55
3	03 15-9-2019	24.74	24.48
4	04 15-9-2019	24.80	24.38
5	05 15-9-2019	24.87	24.40
6	06 15-9-2019	24.71	24.41
7	07 15-9-2019	25.03	24.91
8	08 15-9-2019	25.75	25.85
9	09 15-9-2019	26.64	26.79
10	10 15-9-2019	27.68	27.53
11	11 15-9-2019	28.43	28.98
12	12 15-9-2019	28.29	29.24
13	13 15-9-2019	28.00	30.66
14	14 15-9-2019	27.67	30.97
15	15 15-9-2019	27.29	30.59
16	16 15-9-2019	27.29	29.13
17	17 15-9-2019	26.69	28.72

Các cột **Hà Nội, Vinh, Đà Nẵng, Nha Trang** có chứa dữ liệu missing đã bị loại bỏ

b. Xử lý missing data

df.fillna(value) → thay thế bằng một giá trị cố định

```
1 #PHƯƠNG PHÁP 2: Thay thế (Imputation)
2 #2.1) Thay thế các dữ liệu mất mát bằng một hằng số cố định
3 value = 25.0
4 #thay thế các giá trị missing bằng một giá trị cố định Value
5 data_new = data_temp.fillna(value)
6 print(data_new)
```

		time	Ha Noi	Vinh	Da Nang	Nha Trang	Ho Chi Minh	Ca Mau
0	00	15-9-2019	25.65	24.79	24.01	25.06	25.48	24.97
1	01	15-9-2019	25.00	24.21	24.02	24.93	25.16	24.83
2	02	15-9-2019	25.05	23.73	23.89	24.79	24.80	24.55
3	03	15-9-2019	24.79	23.36	23.83	25.00	24.74	24.48
4	04	15-9-2019	24.59	23.05	23.69	24.82	24.80	24.38
5	05	15-9-2019	24.40	25.00	23.52	24.79	24.87	24.40
6	06	15-9-2019	24.38	22.79	23.68	25.10	24.71	24.41
7	07	15-9-2019	26.72	25.61	24.92	26.56	25.03	24.91
8	08	15-9-2019	28.84	26.93	26.51	26.53	25.75	25.85
9	09	15-9-2019	30.29	28.72	27.48	26.95	26.64	26.79
10	10	15-9-2019	25.00	29.97	25.00	25.00	27.68	27.53
11	11	15-9-2019	32.05	28.93	26.86	27.38	28.43	28.98
12	12	15-9-2019	31.31	28.94	26.65	27.47	28.29	29.24
13	13	15-9-2019	30.95	25.00	27.83	27.44	28.00	30.66
14	14	15-9-2019	30.56	30.62	26.49	27.16	27.67	30.97
15	15	15-9-2019	31.13	30.58	26.29	26.68	27.29	30.59
16	16	15-9-2019	30.80	30.20	25.00	26.45	27.29	29.13

b. Xử lý missing data

df.fillna(method='pad') →
thay thế bằng giá trị liền trước

```
1 #PHƯƠNG PHÁP 2: Thay thế (Imputation)
2 #2.2) Thay thế các dữ liệu mất mát bằng giá trị liền trước của nó
3 data_new2 = data_temp.fillna(method='pad')
4 print(data_new2)
```

		time	Ha Noi	Vinh	Da Nang	Nha Trang	Ho Chi Minh	Ca Mau
0	00	15-9-2019	25.65	24.79	24.01	25.06	25.48	24.97
1	01	15-9-2019	25.65	24.21	24.02	24.93	25.16	24.83
2	02	15-9-2019	25.05	23.73	23.89	24.79	24.80	24.55
3	03	15-9-2019	24.79	23.36	23.83	24.79	24.74	24.48
4	04	15-9-2019	24.59	23.05	23.69	24.82	24.80	24.38
5	05	15-9-2019	24.40	23.05	23.52	24.79	24.87	24.40
6	06	15-9-2019	24.38	22.79	23.68	25.10	24.71	24.41
7	07	15-9-2019	26.72	25.61	24.92	26.56	25.03	24.91
8	08	15-9-2019	28.84	26.93	26.51	26.53	25.75	25.85
9	09	15-9-2019	30.29	28.72	27.48	26.95	26.64	26.79
10	10	15-9-2019	30.29	29.97	27.48	26.95	27.68	27.53
11	11	15-9-2019	32.05	28.93	26.86	27.38	28.43	28.98
12	12	15-9-2019	31.31	28.94	26.65	27.47	28.29	29.24
13	13	15-9-2019	30.95	28.94	27.83	27.44	28.00	30.66
14	14	15-9-2019	30.56	30.62	26.49	27.16	27.67	30.97
15	15	15-9-2019	31.13	30.58	26.29	26.68	27.29	30.59
16	16	15-9-2019	30.80	30.20	26.29	26.45	27.29	29.13

b. Xử lý missing data

df.fillna(method='bfill')

→ thay thế bằng giá trị liền sau

```
1 #PHƯƠNG PHÁP 2: Thay thế (Imputation)
2 #2.3) Thay thế các dữ liệu mất mát bằng giá trị liền sau của nó
3 data_new3 = data_temp.fillna(method='bfill')
4 print(data_new3)
```

		time	Ha Noi	Vinh	Da Nang	Nha Trang	Ho Chi Minh	Ca Mau
0	00	15-9-2019	25.65	24.79	24.01	25.06	25.48	24.97
1	01	15-9-2019	25.05	24.21	24.02	24.93	25.16	24.83
2	02	15-9-2019	25.05	23.73	23.89	24.79	24.80	24.55
3	03	15-9-2019	24.79	23.36	23.83	24.82	24.74	24.48
4	04	15-9-2019	24.59	23.05	23.69	24.82	24.80	24.38
5	05	15-9-2019	24.40	22.79	23.52	24.79	24.87	24.40
6	06	15-9-2019	24.38	22.79	23.68	25.10	24.71	24.41
7	07	15-9-2019	26.72	25.61	24.92	26.56	25.03	24.91
8	08	15-9-2019	28.84	26.93	26.51	26.53	25.75	25.85
9	09	15-9-2019	30.29	28.72	27.48	26.95	26.64	26.79
10	10	15-9-2019	32.05	29.97	26.86	27.38	27.68	27.53
11	11	15-9-2019	32.05	28.93	26.86	27.38	28.43	28.98
12	12	15-9-2019	31.31	28.94	26.65	27.47	28.29	29.24
13	13	15-9-2019	30.95	30.62	27.83	27.44	28.00	30.66
14	14	15-9-2019	30.56	30.62	26.49	27.16	27.67	30.97
15	15	15-9-2019	31.13	30.58	26.29	26.68	27.29	30.59
16	16	15-9-2019	30.80	30.20	25.80	26.45	27.29	29.13

b.Xử lý missing data

df.interpolate() → thay thế giá trị bằng giá trị trung bình của giá trị liền trước và liền sau

```
1 #PHƯƠNG PHÁP 2: Thay thế (Imputation)
2 #2.4)Xử lý các giá trị missing theo phương pháp nội suy
3 #Sử dụng hàm interpolate để thay thế giá trị missing với tham số:
4 #Thuật toán nội suy: Tuyến tính (linear)
5 #Hướng nội suy: Tiến lên (forward)
6 data_new4 = data_temp.interpolate(method='linear', limit_direction='forward')
7 print(data_new4)
```

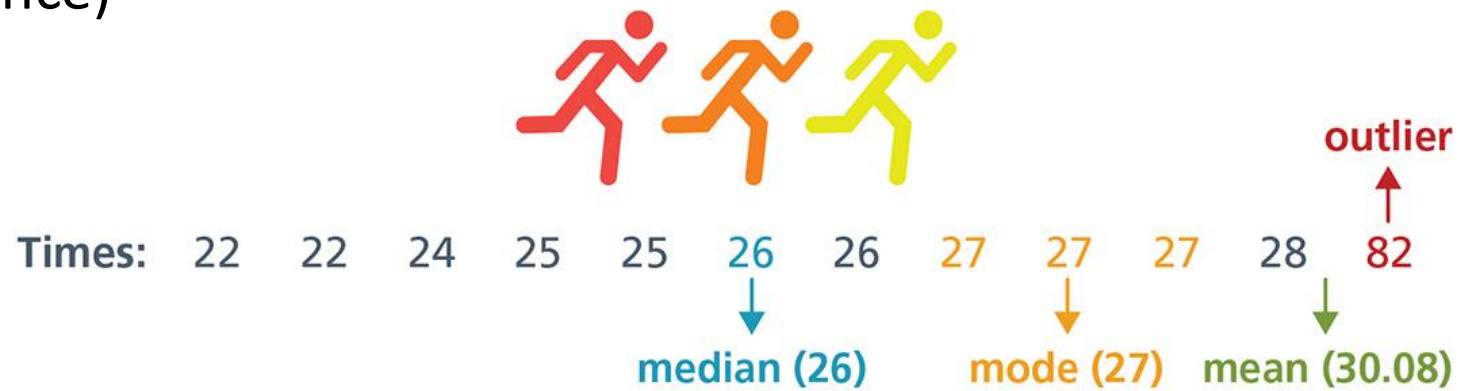
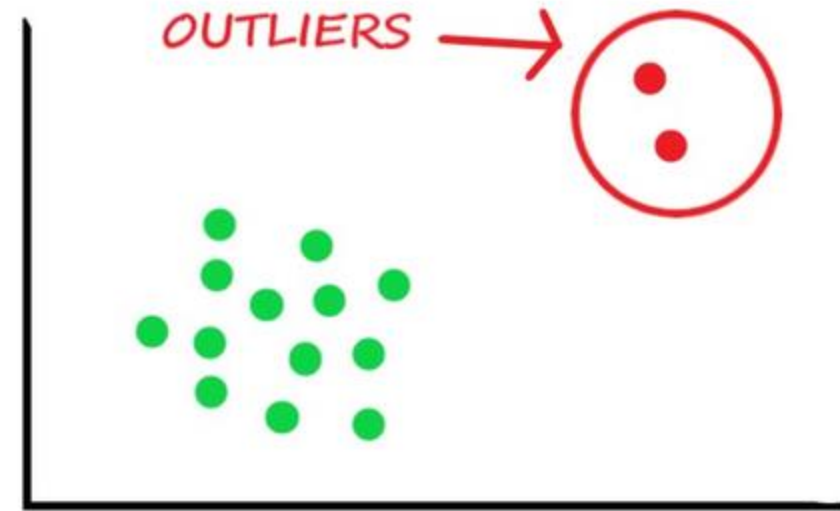
	time	Ha Noi	Vinh	Da Nang	Nha Trang	Ho Chi Minh	Ca Mau
0	00 15-9-2019	25.65	24.79	24.010	25.060	25.48	24.97
1	01 15-9-2019	25.35	24.21	24.020	24.930	25.16	24.83
2	02 15-9-2019	25.05	23.73	23.890	24.790	24.80	24.55
3	03 15-9-2019	24.79	23.36	23.830	24.805	24.74	24.48
4	04 15-9-2019	24.59	23.05	23.690	24.820	24.80	24.38
5	05 15-9-2019	24.40	22.92	23.520	24.790	24.87	24.40
6	06 15-9-2019	24.38	22.79	23.680	25.100	24.71	24.41
7	07 15-9-2019	26.72	25.61	24.920	26.560	25.03	24.91
8	08 15-9-2019	28.84	26.93	26.510	26.530	25.75	25.85
9	09 15-9-2019	30.29	28.72	27.480	26.950	26.64	26.79
10	10 15-9-2019	31.17	29.97	27.170	27.165	27.68	27.53
11	11 15-9-2019	32.05	28.93	26.860	27.380	28.43	28.98
12	12 15-9-2019	31.31	28.94	26.650	27.470	28.29	29.24
13	13 15-9-2019	30.95	29.78	27.830	27.440	28.00	30.66
14	14 15-9-2019	30.56	30.62	26.490	27.160	27.67	30.97
15	15 15-9-2019	31.13	30.58	26.290	26.680	27.29	30.59
16	16 15-9-2019	30.80	30.20	26.045	26.450	27.29	29.13

9. Phát hiện và xử lý ngoại lai

Giá trị ngoại lai (Outliers)

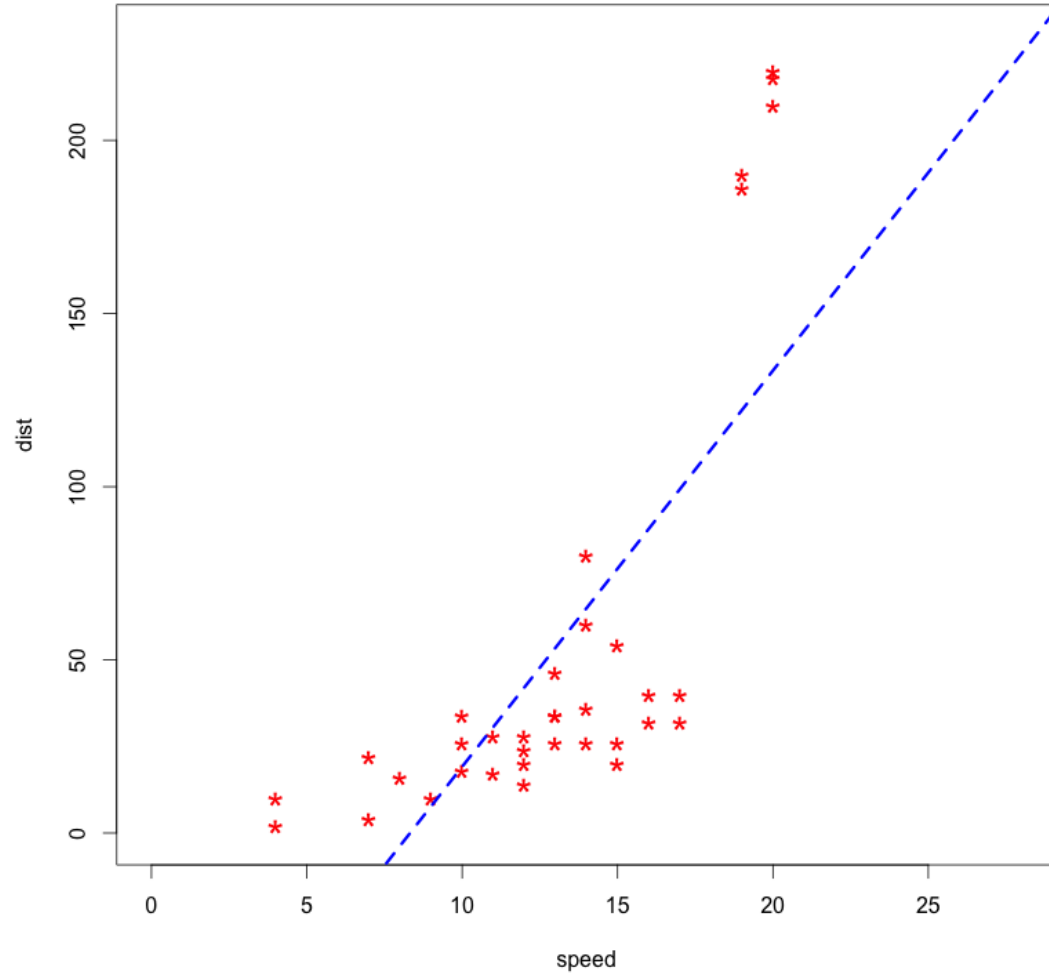
Một điểm ngoại lai là một điểm dữ liệu khác biệt đáng kể so với phần còn lại của tập dữ liệu. Ta thường xem các giá trị ngoại lai như là các mẫu dữ liệu đặc biệt, cách xa khỏi phần lớn dữ liệu khác trong tập dữ liệu

- Hệ thống phát hiện xâm nhập (Intrusion detection systems)
- Phát hiện gian lận tín dụng (Credit card fraud)
- Trong chuẩn đoán y tế (Medical diagnosis)
- Trong thực thi pháp luật (Law enforcement)
- Trong khoa học trái đất (Earth science)

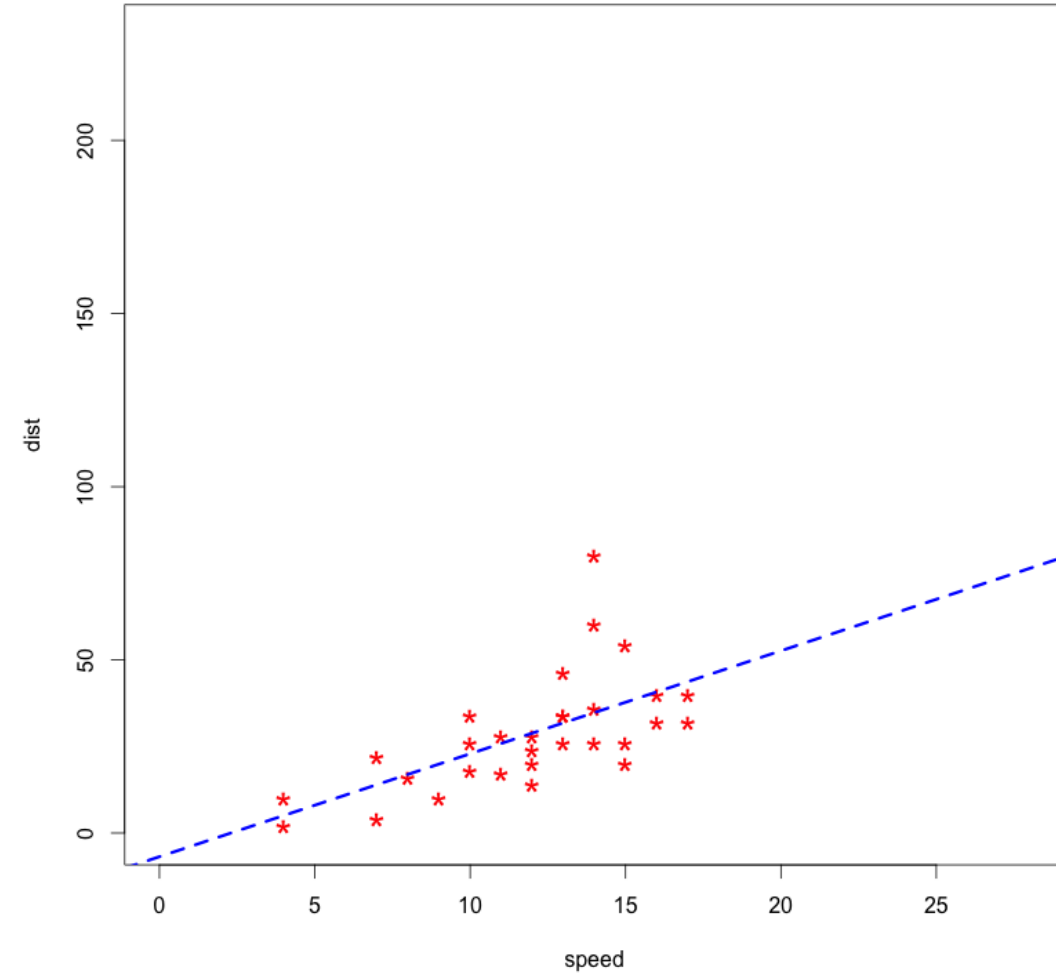


Giá trị ngoại lai (Outliers)

With Outliers



Outliers removed
A much better fit!



Giá trị ngoại lai (Outliers)

Outliers

How to
identify?

Box plot

Interquartile Range (IQR) based method

Standard Deviation based method

Histogram

How to
handle?

Doing nothing

Deleting/Trimming

Winsorizing

Transformation

Binning

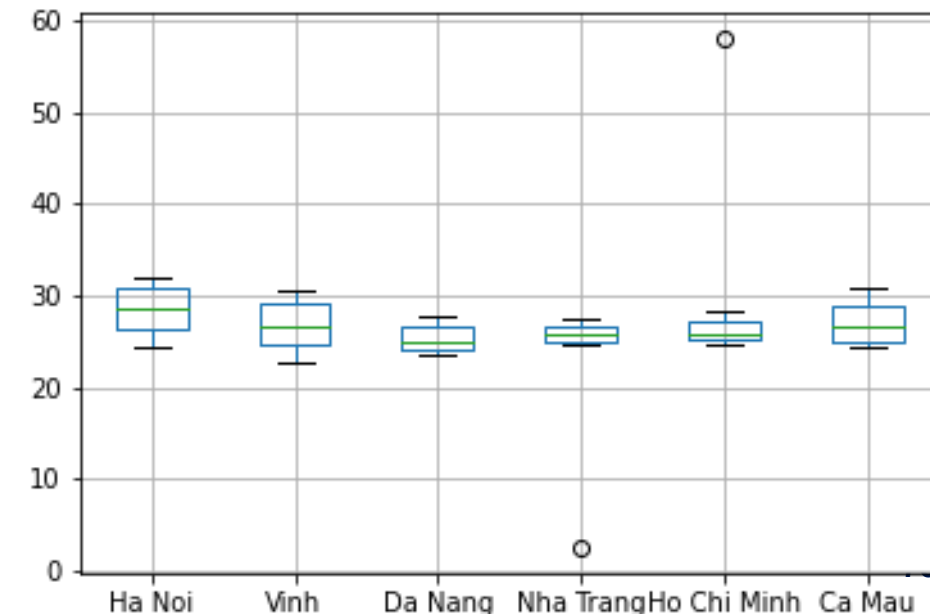
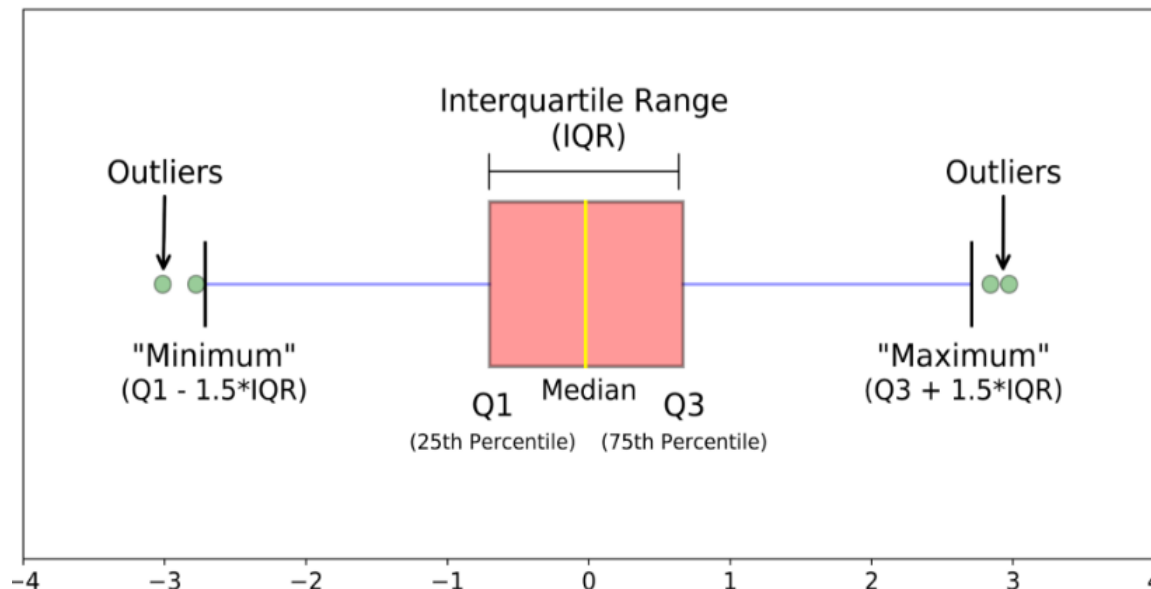
Use robust estimators

Imputing

Phát hiện Outliers

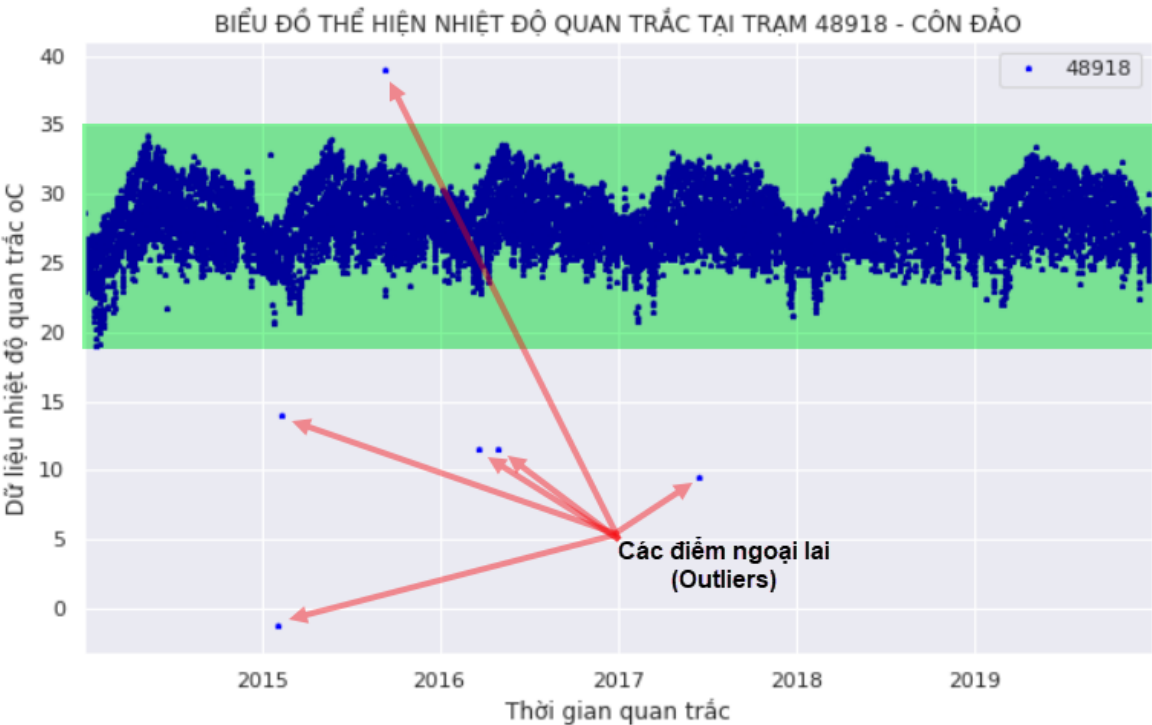
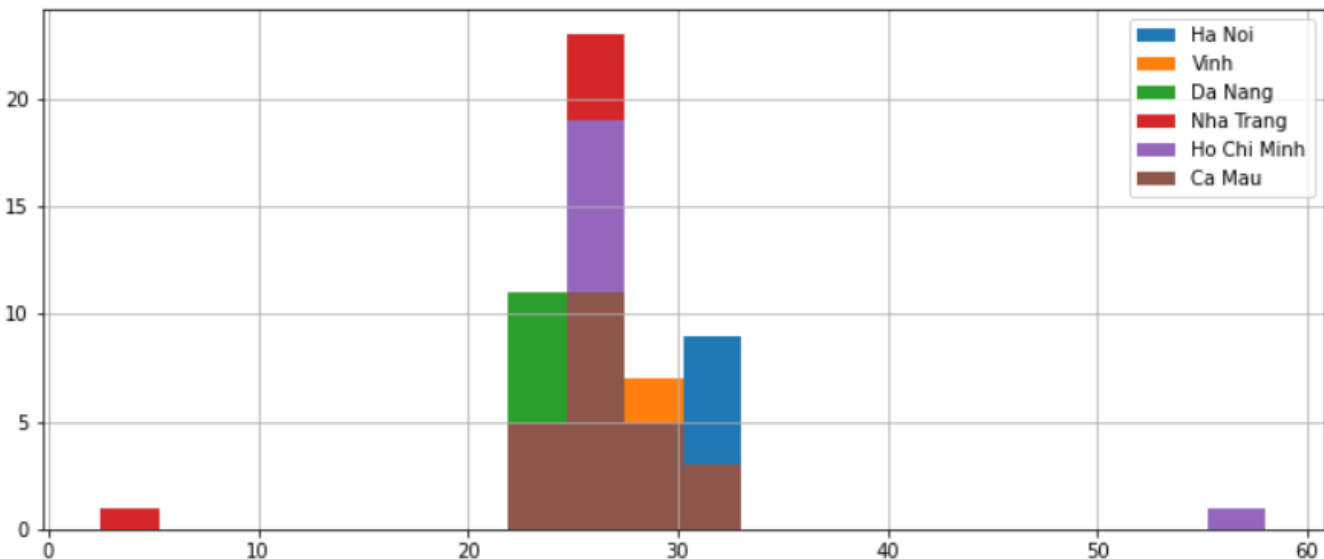
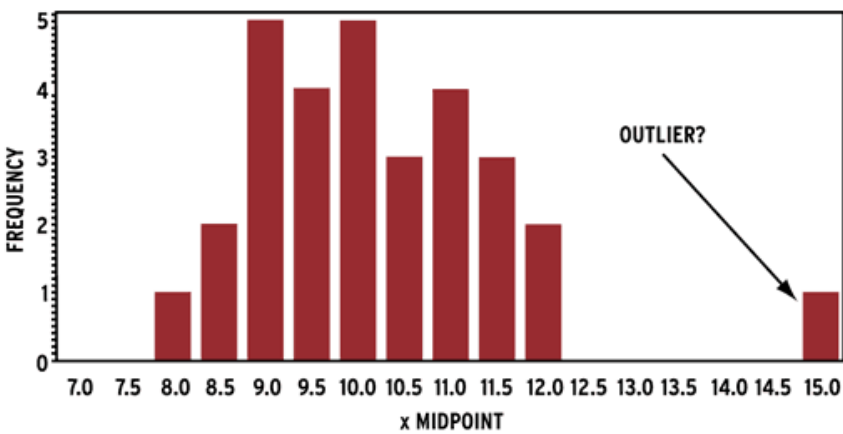
Biểu đồ Box-plot được sử dụng để đo khuynh hướng phân tán và xác định các giá trị ngoại lai của tập dữ liệu

- Giá trị bé nhất (**Minimum**) của tập dữ liệu được xác định bằng $Q1 - 1.5 * IQR$;
- Tứ phân vị thứ nhất (**Q1**) của tập dữ liệu
- Tứ phân vị thứ hai (**Q2**) chính là giá trị trung vị (Median) của tập dữ liệu
- Tứ phân vị thứ ba (**Q3**) của tập dữ liệu
- Giá trị lớn nhất (**Maximum**) của tập dữ liệu có giá trị bằng $Q3 + 1.5 * IQR$



Phát hiện Outliers

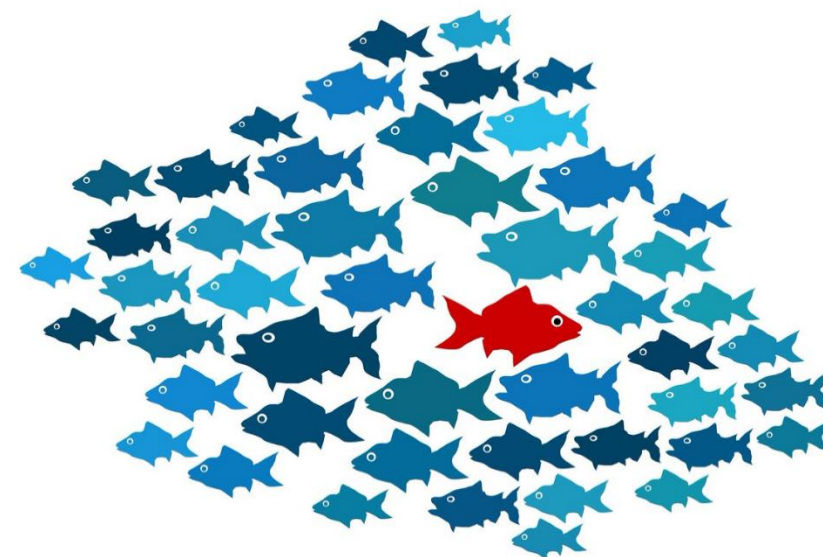
Sử dụng biểu đồ histogram;
Hoặc scatter (nhiều hơn 2 biến).



Xử lý Outliers

Không có một phương pháp, cách thức xử lý ngoại lai chung nào áp dụng cho tất cả các bài toán, các kiểu dữ liệu khác nhau.

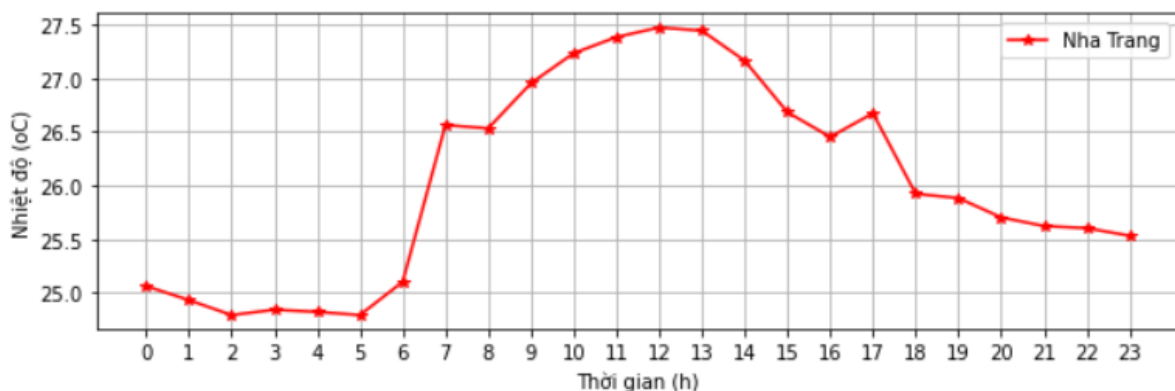
- Loại bỏ dòng dữ liệu chứa ngoại lai.
- Thay thế bằng một giá trị khác
- Thay thế giá trị ngoại lai về NAN (null) coi như là một điểm dữ liệu missing và xử lý như với dữ liệu missing



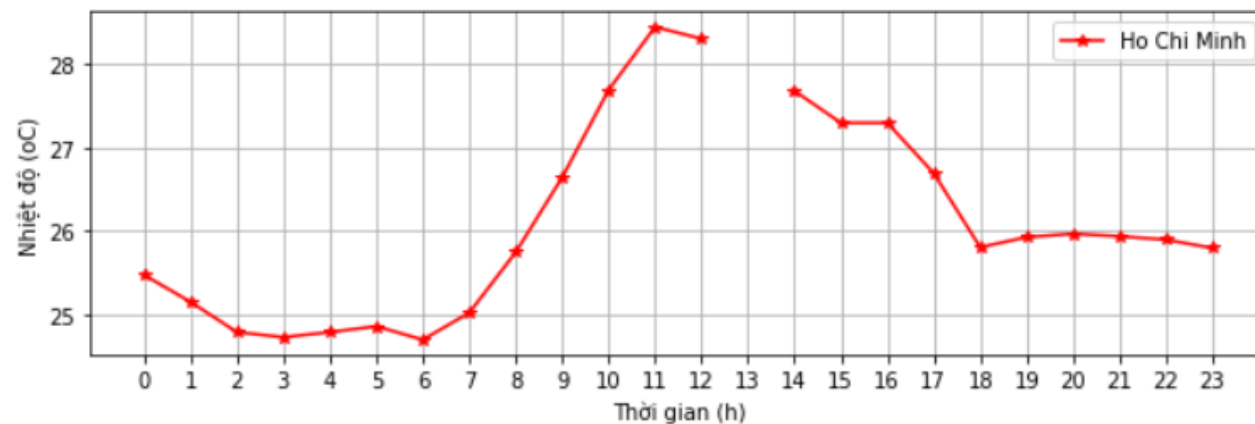
Để lựa chọn được phương pháp phù hợp cần có những hiểu biết sâu sắc về tập dữ liệu, về bài toán đang giải quyết, có thể sử dụng chỉ một phương pháp xử lý ngoại lai và/hoặc kết hợp cả 3 nhóm phương pháp đã chỉ ra ở trên để xử lý ngoại lai cho cùng một tập dữ liệu.

Xử lý Outliers

```
1 #Với giá trị ngoại lai trạm Nha trang
2 #Xử lý bằng cách thay thế giá trị: 2.51 --> 25.1
3 x = np.arange(0,24)
4 plt.rcParams["figure.figsize"] = (10,3)
5 data_handling_outlier.loc[6,'Nha Trang'] = 25.10
6 data_handling_outlier[['Nha Trang']].plot(style='-*', color='red')
7 plt.xticks(x)
8 plt.xlabel('Thời gian (h)')
9 plt.ylabel('Nhiệt độ (oC)')
10 plt.grid(True)
11 plt.show()
```



```
1 #Với giá trị ngoại lai trạm Hồ Chí Minh
2 #Xử lý bằng cách chuyển về giá trị Null - Coi như giá trị missing
3 data_handling_outlier.loc[13,'Ho Chi Minh'] = np.NaN
4 data_handling_outlier[['Ho Chi Minh']].plot(style='-*', color='red')
5 plt.xticks(x)
6 plt.xlabel('Thời gian (h)')
7 plt.ylabel('Nhiệt độ (oC)')
8 plt.grid(True)
9 plt.show()
```



10. Phân tích dữ liệu bán hàng

(Tiếp cận từ bài toán thực tế)

THE DATA ANALYSIS PROCESS



Bước 1: Xác định bài toán

Mục tiêu của dự án

Công ty bao gồm một chuỗi 10 cửa hàng kinh doanh đồ điện tử, được đặt tại 10 thành phố của 8 bang nước Mỹ.

1. Austin (TX - Texas)
2. Dallas (TX - Texas)
3. San Francisco (CA - California)
4. Los Angeles (CA - California)
5. New York City (NY - New York)
6. Portland (ME - Maine)
7. Portland (OR - Oregon)
8. Atlanta (GA - Georgia)
9. Seattle (WA - Washington)
10. Boston (MA - Massachusetts)



Mục tiêu của dự án

Mục tiêu của dự án:

- Đánh giá hoạt động kinh doanh của chuỗi cửa hàng trong năm 2024
- Thực hiện phân tích tìm ra các thông tin có ích (insights) từ dữ liệu để cải thiện, nâng cao hoạt động kinh doanh tại các cửa hàng.

Trả lời các câu hỏi:

1. **Câu hỏi 1:** *Cửa hàng ở thành phố nào hoạt động hiệu quả nhất?*
2. **Câu hỏi 2:** *Tháng nào trong năm có doanh số bán hàng cao nhất - thấp nhất?*
3. **Câu hỏi 3:** *Khách hàng mua sản phẩm thường tập trung vào khung giờ nào trong ngày?*
4. **Câu hỏi 4:** *Khách hàng thường mua các Sản phẩm nào cùng với nhau?*
5. **Câu hỏi 5:** *Sản phẩm nào bán được số lượng nhiều nhất? Tại sao?*

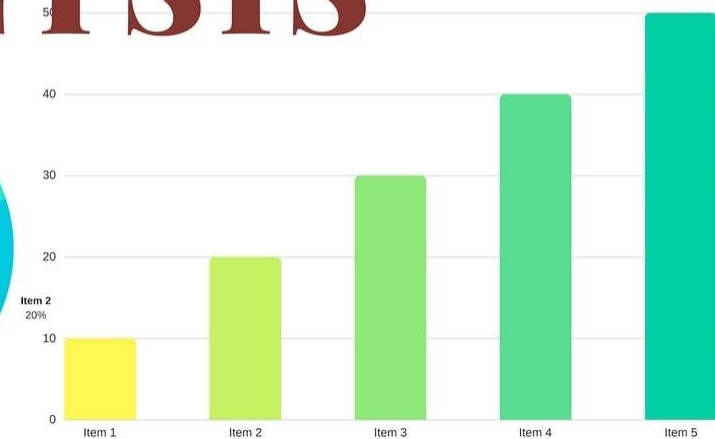
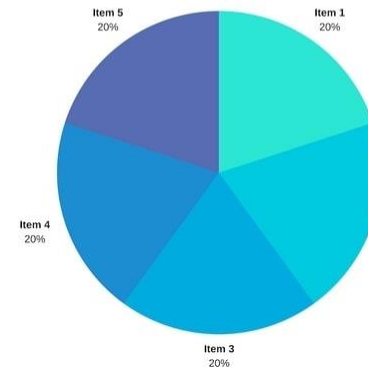
Bước 2: Thu thập dữ liệu

Thu thập dữ liệu

Dữ liệu bao gồm 12 file của một chuỗi cửa hàng kinh doanh thiết bị điện tử; Mỗi file dữ liệu lưu lại toàn bộ các đơn hàng đã bán được theo từng tháng tương ứng của năm 2024.

- * Dữ liệu tháng 1: Sales_01_2024.xlsx
- * Dữ liệu tháng 2: Sales_02_2024.xlsx
- * Dữ liệu tháng 3: Sales_03_2024.xlsx
- * Dữ liệu tháng 4: Sales_04_2024.xlsx
- * Dữ liệu tháng 5: Sales_05_2024.xlsx
- * Dữ liệu tháng 6: Sales_06_2024.xlsx
- * Dữ liệu tháng 7: Sales_07_2024.xlsx
- * Dữ liệu tháng 8: Sales_08_2024.xlsx
- * Dữ liệu tháng 9: Sales_09_2024.xlsx
- * Dữ liệu tháng 10: Sales_10_2024.xlsx
- * Dữ liệu tháng 11: Sales_11_2024.xlsx
- * Dữ liệu tháng 12: Sales_12_2024.xlsx

SALES ANALYSIS



Thu thập dữ liệu

Mỗi file dữ liệu bao gồm 6 thông tin:

- 1. Order ID: Mã đơn hàng
- 2. Product: Tên sản phẩm đã bán theo từng đơn hàng
- 3. Quantity Ordered: Số lượng sản phẩm bán
- 4. Price Each: Giá của mỗi sản phẩm
- 5. Order Date: Thời gian bán hàng
- 6. Purchase Address: Địa chỉ mua hàng



Order ID	Product	Quantity Ordered	Price Each	Order Date	Purchase Address
141234	iPhone	1	700	01/22/19 21:25	944 Walnut St, Boston, MA 02215
141235	Lightning Charging Cable	1	14.95	01/28/19 14:15	185 Maple St, Portland, OR 97035
141236	Wired Headphones	2	11.99	01/17/19 13:33	538 Adams St, San Francisco, CA 94016
141237	27in FHD Monitor	1	149.99	01/05/19 20:33	738 10th St, Los Angeles, CA 90001
141238	Wired Headphones	1	11.99	01/25/19 11:59	387 10th St, Austin, TX 73301

(Nếu khách hàng mua nhiều sản phẩm, có nhiều bản ghi với cùng mã Order ID)

Bước 3: Chuẩn bị dữ liệu

Chuẩn bị dữ liệu

Tập dữ liệu tổng hợp là tập dữ liệu thô cần được chuẩn hóa và làm sạch:

1. Tích hợp dữ liệu
2. Phát hiện và xử lý dữ liệu thiếu (missing)
3. Phát hiện và xử lý dữ liệu trùng lặp
4. Chuyển đổi kiểu dữ liệu các cột cho phù hợp
5. Bổ sung thêm các cột dữ liệu mới phục vụ phân tích
6. Lưu dữ liệu sau xử lý



Chuẩn bị dữ liệu

Tập dữ liệu sau khi được chuẩn hóa và làm sạch:

Order ID	Product	Quantity Ordered	Price Each	Order Date	Purchase Address	Month	Hour	Sales	City
147268	Wired Headphones	1	11.99	2024-01-01 03:07:00	9 Lake St, New York City, NY 10001	1	3	11.99	New York City (NY)
148041	USB-C Charging Cable	1	11.95	2024-01-01 03:40:00	760 Church St, San Francisco, CA 94016	1	3	11.95	San Francisco (CA)
149343	Apple Airpods Headphones	1	150.0	2024-01-01 04:56:00	735 5th St, New York City, NY 10001	1	4	150.0	New York City (NY)
149964	AAA Batteries (4-pack)	1	2.99	2024-01-01 05:53:00	75 Jackson St, Dallas, TX 75001	1	5	2.99	Dallas (TX)
149350	USB-C Charging Cable	2	11.95	2024-01-01 06:03:00	943 2nd St, Atlanta, GA 30301	1	6	23.9	Atlanta (GA)
141732	iPhone	1	700.0	2024-01-01 06:13:00	446 Pine St, Atlanta, GA 30301	1	6	700.0	Atlanta (GA)
149620	Lightning Charging Cable	1	14.95	2024-01-01 06:34:00	338 Chestnut St, San Francisco, CA 94016	1	6	14.95	San Francisco (CA)
142451	AAA Batteries (4-pack)	1	2.99	2024-01-01 06:41:00	232 12th St, Boston, MA 02215	1	6	2.99	Boston (MA)
146039	34in Ultrawide Monitor	1	379.99	2024-01-01 07:24:00	53 River St, San Francisco, CA 94016	1	7	379.99	San Francisco (CA)
143498	AA Batteries (4-pack)	3	3.84	2024-01-01 07:26:00	428 Highland St, New York City, NY 10001	1	7	11.52	New York City (NY)
141316	AAA Batteries (4-pack)	3	2.99	2024-01-01 07:26:00	235 South St, Seattle, WA 98101	1	7	8.97	Seattle (WA)
144804	Wired Headphones	1	11.99	2024-01-01 07:29:00	628 Lake St, New York City, NY 10001	1	7	11.99	New York City (NY)
144804	iPhone	1	700.0	2024-01-01 07:29:00	628 Lake St, New York City, NY 10001	1	7	700.0	New York City (NY)
145270	Google Phone	1	600.0	2024-01-01 07:33:00	392 4th St, Dallas, TX 75001	1	7	600.0	Dallas (TX)
142789	AAA Batteries (4-pack)	1	2.99	2024-01-01 07:35:00	336 Spruce St, Boston, MA 02215	1	7	2.99	Boston (MA)
148088	Wired Headphones	1	11.99	2024-01-01 07:37:00	199 Spruce St, San Francisco, CA 94016	1	7	11.99	San Francisco (CA)
141364	AA Batteries (4-pack)	1	3.84	2024-01-01 07:46:00	657 Lincoln St, Dallas, TX 75001	1	7	3.84	Dallas (TX)
149586	USB-C Charging Cable	1	11.95	2024-01-01 07:47:00	631 Lakeview St, San Francisco, CA 94016	1	7	11.95	San Francisco (CA)
145878	Macbook Pro Laptop	1	1700.0	2024-01-01 07:48:00	456 5th St, Los Angeles, CA 90001	1	7	1700.0	Los Angeles (CA)
146056	Lightning Charging Cable	1	14.95	2024-01-01 07:53:00	622 Hill St, Portland, OR 97035	1	7	14.95	Portland (OR)
144994	ThinkPad Laptop	1	999.99	2024-01-01 08:13:00	903 Willow St, Los Angeles, CA 90001	1	8	999.99	Los Angeles (CA)
142156	34in Ultrawide Monitor	1	379.99	2024-01-01 08:16:00	540 2nd St, San Francisco, CA 94016	1	8	379.99	San Francisco (CA)
150367	Lightning Charging Cable	1	14.95	2024-01-01 08:25:00	576 West St, Los Angeles, CA 90001	1	8	14.95	Los Angeles (CA)
149421	Wired Headphones	1	11.99	2024-01-01 08:31:00	920 Willow St, Boston, MA 02215	1	8	11.99	Boston (MA)

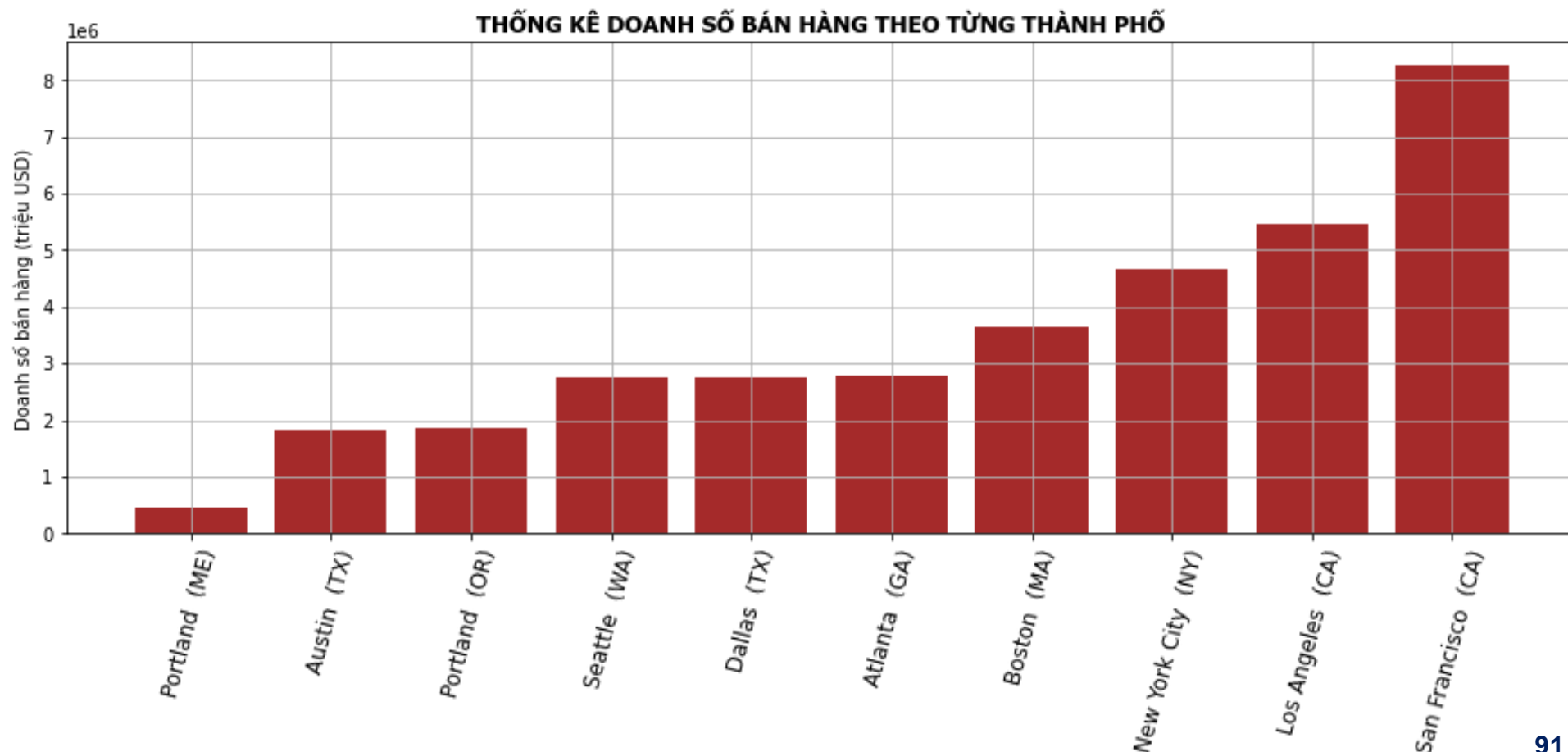
Bước 4+5: Phân tích, trực quan hóa và chia sẻ kết quả

Phân tích – Chia sẻ kết quả

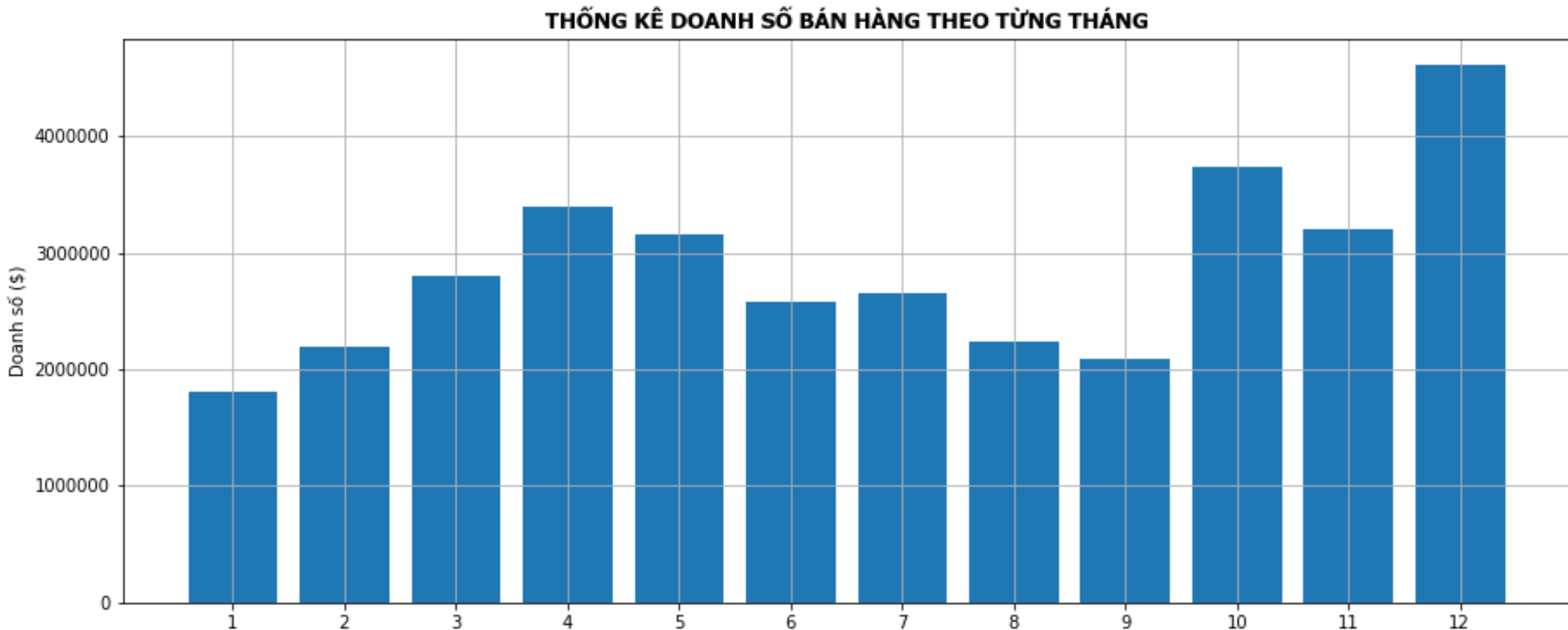


Câu hỏi 1: Cửa hàng ở thành phố nào bán được hàng nhiều nhất?

Sales	
City	
Portland (ME)	449321.38
Austin (TX)	1817544.35
Portland (OR)	1869857.57
Seattle (WA)	2744896.03
Dallas (TX)	2763659.01
Atlanta (GA)	2794199.07
Boston (MA)	3657300.76
New York City (NY)	4660526.52
Los Angeles (CA)	5447304.29
San Francisco (CA)	8252258.67

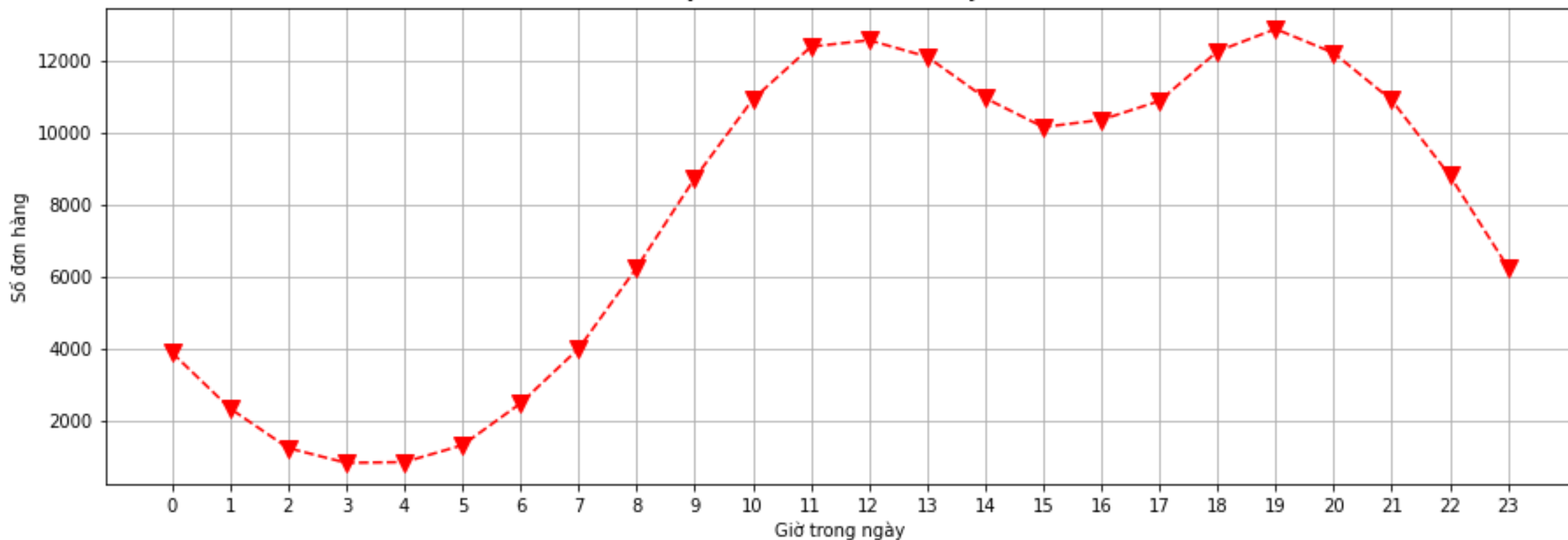


Câu hỏi 2: Tháng nào trong năm có doanh số bán hàng cao nhất - thấp nhất?



Câu hỏi 3: Khách hàng mua sản phẩm thường tập trung vào khung giờ nào trong ngày?

THỐNG KÊ SỐ LƯỢNG SẢN PHẨM BÁN ĐƯỢC THEO TỪNG GIỜ



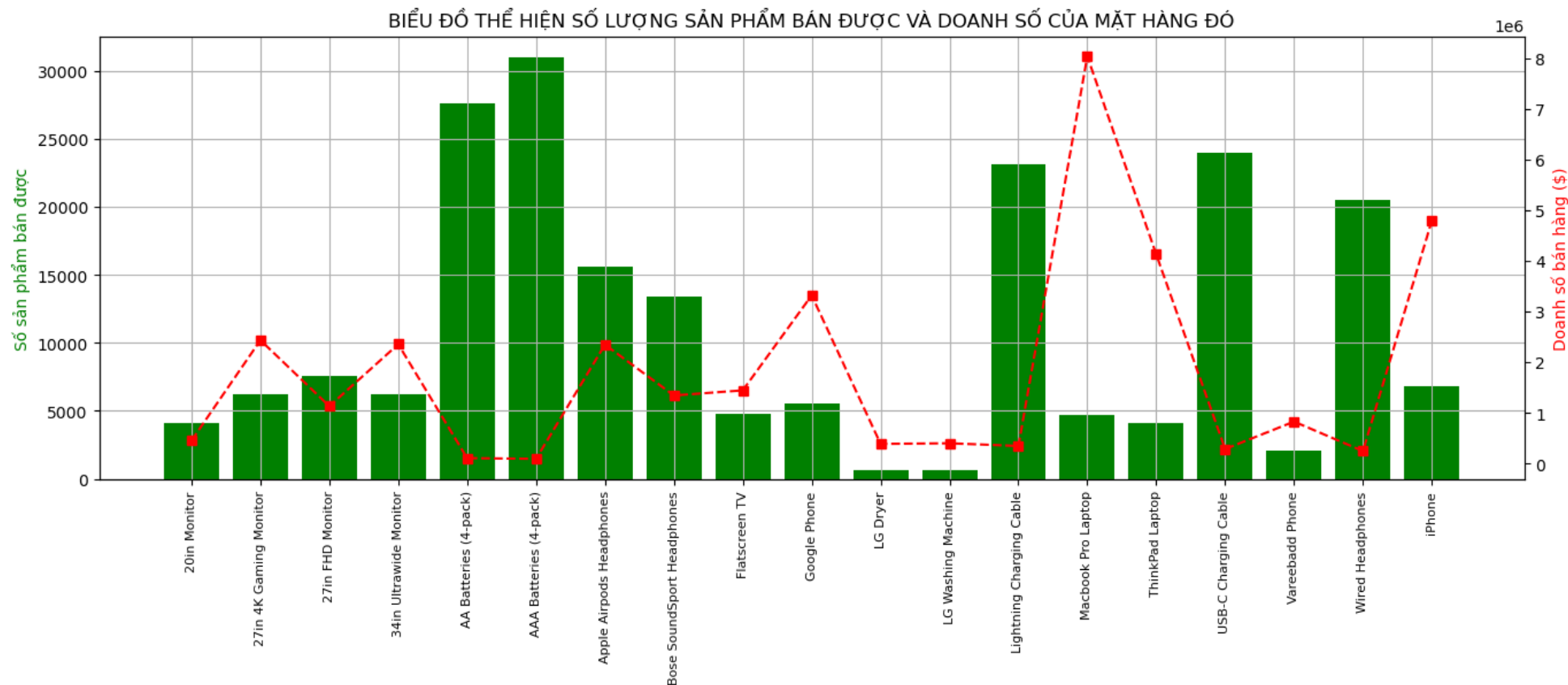
Phân tích – Chia sẻ kết quả

Câu hỏi 4: Trong các hóa đơn của Khách hàng, Sản phẩm nào thường được mua cùng với nhau?



```
('iPhone', 'Lightning Charging Cable') 1010
('Google Phone', 'USB-C Charging Cable') 997
('iPhone', 'Wired Headphones') 462
('Google Phone', 'Wired Headphones') 422
('iPhone', 'Apple AirPods Headphones') 372
('Vareebadd Phone', 'USB-C Charging Cable') 368
('Google Phone', 'Bose SoundSport Headphones') 228
('Wired Headphones', 'USB-C Charging Cable') 203
('Vareebadd Phone', 'Wired Headphones') 149
('Lightning Charging Cable', 'Wired Headphones') 129
('Apple AirPods Headphones', 'Lightning Charging Cable') 116
('Lightning Charging Cable', 'AA Batteries (4-pack)') 106
('Bose SoundSport Headphones', 'USB-C Charging Cable') 102
('Apple AirPods Headphones', 'Wired Headphones') 100
('Lightning Charging Cable', 'USB-C Charging Cable') 100
```

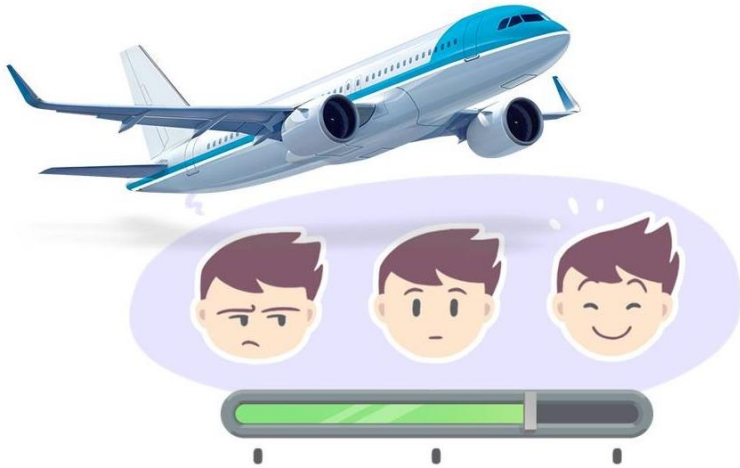
Câu hỏi 5: Sản phẩm nào của chuỗi cửa hàng bán được số lượng nhiều nhất? Tại sao?



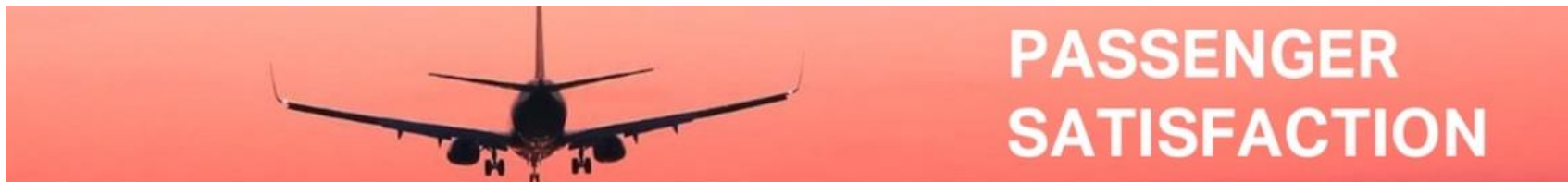
BÀI TẬP THỰC HÀNH TỔNG HỢP

Mô tả bài toán

Nhằm cải thiện, nâng cao chất lượng phục vụ hành khách. Giúp tìm ra những dịch vụ, những khâu còn hạn chế để đem đến sự hài lòng cho hành khách qua đó thu hút và giữ chân hành khách sử dụng dịch vụ bay của hãng mình.



Hãng hàng không thực hiện một dự án về khai phá dữ liệu (Data Mining), để xác định những dịch vụ mà hãng mình cung cấp, hành khách phản hồi đánh giá mức độ hài lòng ra sao.



Mô tả bài toán

Để thực hiện được dự án này, Cần tiến hành lấy khảo sát của tất cả hành khách đã sử dụng dịch vụ bay của hãng thông qua các kênh khác nhau (khảo sát Online, sử dụng các phiếu khảo sát, phỏng vấn), thu thập và tổng hợp các dữ liệu này lại để phục vụ phân tích.



quicktapsurvey.com

Airline Passenger Satisfaction

Your logo here

11 Please rate the following criteria based on your experience with our airline:

	Very Poor	Poor	Neutral	Good	Excellent
Luggage space	☐	☐	☐	☐	☐
Seating comfort	☐	☐	☐	☐	☐
Aircraft cleanliness	☐	☐	☐	☐	☐

12 Overall, how would you rate your interaction with our airline during your flight?

☐ ☐ ☐ ☐ ☐

13 Do you participate in an airline rewards program?

START PLANETS

CUSTOMER FEEDBACK CARD

Name : _____ Date : _____

Email : _____ Phone : _____

Dear customer give us a small feedback About our Airlines Aviation services And suggest us new Services which is best for our Services

How is our Airlines Aviation Services

Good : ☐ Very Good : ☐ Poor : ☐

How is Our Airlines Aviation Services performance

Good : ☐ Very Good : ☐ Poor : ☐

How is our Staff performance

Good : ☐ Very Good : ☐ Poor : ☐

How would you rate your overall experience with our service?

Good : ☐ Very Good : ☐ Poor : ☐

how satisfied are you with our customer service representative?

Good : ☐ Very Good : ☐ Poor : ☐

Comments and suggestions: _____

THANK YOU

Mô tả tập dữ liệu



Dữ liệu của dự án bao gồm 2 loại:

- **File 1: Data_Passenger_Airlines.csv** - Cho biết thông tin về hành khách đi trên các chuyến bay.
- **File 2: Data_Feedback_Month.Month.xlsx** - Đánh giá của hành khách về các dịch vụ bay của hãng và cảm nhận của hành khách sau chuyến bay.

Mô tả tập dữ liệu

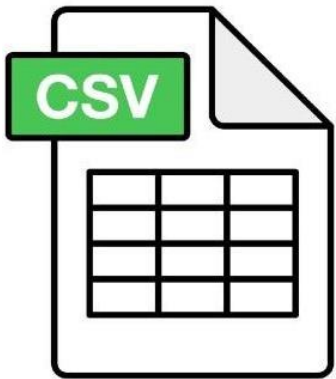
Dữ liệu hành khách đi trên các chuyến bay: Lưu trữ trong file csv **Data_Passenger_Airlines.csv**

Mỗi khách hàng bao gồm các thông tin:



1. **Code:** Mã khách hàng
2. **Gender:** Giới tính của hành khách (Male|Female)
3. **Customer Type:** Loại hành khách.
 - * Khách hàng thân thiết (Loyal Customer): Khách hàng đã đi nhiều lần (≥ 2) với hãng
 - * Khách hàng không thân thiết (Disloyal Customer): Khách hàng không thân thiết (sử dụng dịch vụ bay lần đầu của hãng)
4. **Age:** Tuổi của khách hàng
5. **Type of Travel:** Mục đích đi máy bay
 - * Đi với mục đích công việc (Business travel)
 - * Đi với mục đích cá nhân (Personal Travel)
6. **Class:** Loại vé (Business | Eco Plus | Eco)
7. **Flight Distance:** Khoảng cách di chuyển của chuyến bay (Mile)
8. **Departure Delay in Minutes:** Thời gian trễ của chuyến bay tại ga xuất phát (Phút)
9. **Arrival Delay in Minutes:** Thời gian trễ của chuyến bay tại ga đến (phút)

Mô tả tập dữ liệu



Code	Gender	Customer Type	Age	Type of Travel	Class	Flight Distance	Departure Delay in Minutes	Arrival Delay in Minutes
WX02553023	Male	Loyal Customer	13	Personal Travel	Eco Plus	460	25.0	18.0
QS99844747	Male	Disloyal Customer	25	Business travel	Business	235	1.0	6.0
OL00432935	Female	Loyal Customer	26	Business travel	Business	1142	0.0	0.0
CM55347278	Female	Loyal Customer	25	Business travel	Business	562	11.0	9.0
TZ46480191	Male	Loyal Customer	61	Business travel	Business	214	0.0	0.0
WG85023638	Female	Loyal Customer	26	Personal Travel	Eco	1180	0.0	0.0
CS15349553	Male	Loyal Customer	47	Personal Travel	Eco	1276	9.0	23.0
IX94937318	Female	Loyal Customer	52	Business travel	Business	2035	4.0	0.0
HI93389362	Female	Loyal Customer	41	Business travel	Business	853	0.0	0.0
TM51865878	Male	Disloyal Customer	20	Business travel	Eco	1061	0.0	0.0
RH39935810	Female	Disloyal Customer	24	Business travel	Eco	1182	0.0	0.0
SC38759946	Female	Loyal Customer	12	Personal Travel	Eco Plus	308	0.0	0.0
JN42885112	Male	Loyal Customer	53	Business travel	Eco	834	28.0	8.0
RX25535201	Male	Loyal Customer	33	Personal Travel	Eco	946	0.0	0.0
MS27111818	Female	Loyal Customer	26	Personal Travel	Eco	453	43.0	35.0
TM98090585	Male	Disloyal Customer	13	Business travel	Eco	486	1.0	0.0
LN32495921	Female	Loyal Customer	26	Business travel	Business	2123	49.0	51.0
TG00760132	Male	Loyal Customer	41	Business travel	Business	2075	0.0	10.0
DV25875314	Female	Loyal Customer	45	Business travel	Business	2486	7.0	5.0
MY26067427	Male	Loyal Customer	38	Personal Travel	Eco	460	17.0	18.0
QH99609545	Male	Loyal Customer	9	Business travel	Eco	1174	0.0	4.0

Mô tả tập dữ liệu

Dữ liệu đánh giá các dịch vụ của hành khách: Lưu trữ trong file excel **Data_Feedback_Month.Month.xlsx** với 12 sheets tương ứng với dữ liệu khảo sát của 12 tháng trong năm; Bao gồm các thông tin sau:

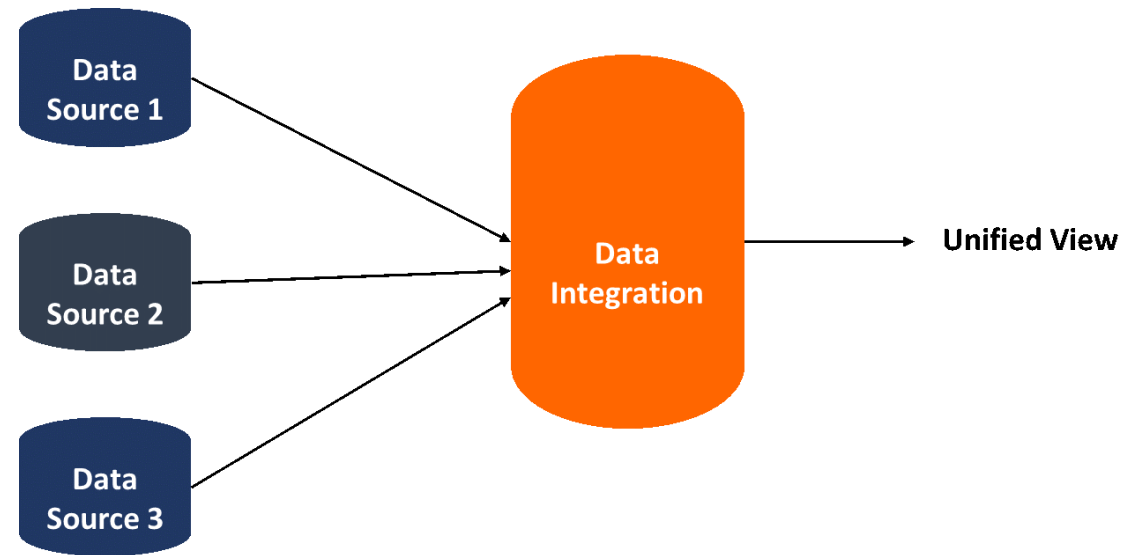
1. **Code:** Mã khách hàng
2. **Inflight wifi service:** Dịch vụ wifi trên chuyến bay (0:Thấp nhất – 5: Cao nhất)
3. **Departure/Arrival time convenient:** Mức độ thuận tiện trong thời gian ở Ga khởi hành| Ga đến (0-5)
4. **Ease of Online booking:** Mức độ dễ dàng khi đặt vé Online (0-5)
5. **Food and drink:** Đồ ăn và đồ uống phục vụ trên chuyến bay (0-5)
6. **Seat comfort:** Sự thoải mái của chỗ ngồi (0-5)
7. **Inflight entertainment:** Dịch vụ giải trí trên chuyến bay (0-5)
8. **Baggage handling:** Vấn đề liên quan đến hành lý xách tay (0-5)
9. **Checkin service:** Dịch vụ checkin (0-5)
10. **Inflight service:** Dịch vụ trên chuyến bay (0-5)
11. **Cleanliness:** Vệ sinh trên chuyến bay (0-5)
12. **satisfaction:** Tổng quan chung về mức độ hài lòng
 - * KHÔNG HÀI LÒNG với chuyến bay (neutral or dissatisfied)
 - * HÀI LÒNG với chuyến bay (satisfied)



Yêu cầu:

Yêu cầu 1. Tích hợp dữ liệu:

Dữ liệu liên quan đến dự án đang được lưu trữ ở 2 file với 2 định dạng khác nhau; Hãy tìm hiểu và tích hợp 2 file dữ liệu thành 1 file dữ liệu duy nhất (Dataset) để phục vụ cho việc phân tích.



Yêu cầu:

Yêu cầu 2. Khám phá để hiểu tập dữ liệu (Understanding)

Tìm hiểu dữ liệu của các thuộc tính trong Dataset đã tích hợp hãy xác định các đặc trưng thống kê:

- Với các thuộc tính dữ liệu định lượng: Xác định các tham số Min, Max, Mean, Median, standard deviation (std), Q1, Q2, Q3.
- Với các thuộc tính dữ liệu định tính: Xác định số các giá trị khác nhau, giá trị xuất hiện nhiều nhất (mode), số lần xuất hiện.

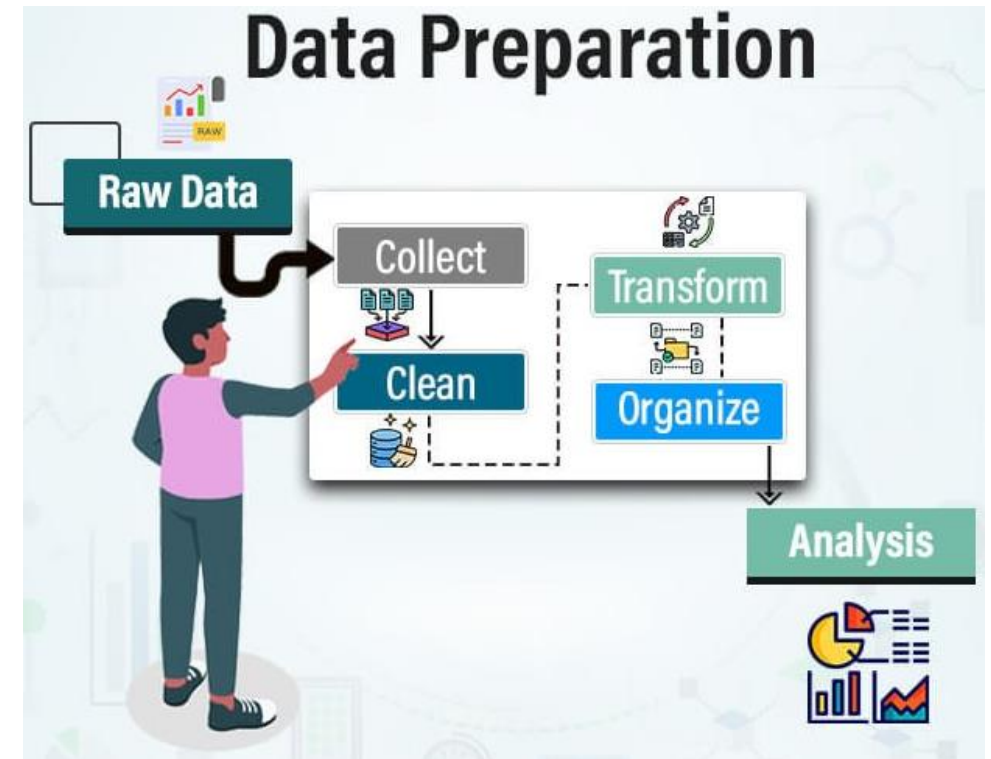
	count	mean	std	min	25%	50%	75%	max
Age	129894.0	39.443708	16.058282	7.0	27.0	40.0	51.0	1990.0
Flight Distance	129894.0	1190.218147	997.407838	31.0	414.0	844.0	1744.0	4983.0
Departure Delay in Minutes	129894.0	14.713797	38.069224	0.0	0.0	0.0	12.0	1592.0
Arrival Delay in Minutes	129503.0	15.088083	38.465143	-100.0	0.0	0.0	13.0	1584.0
Inflight wifi service	129893.0	2.386695	1.551981	0.0	1.0	2.0	4.0	5.0
Departure/Arrival time convenient	129893.0	3.057701	1.526757	0.0	2.0	3.0	4.0	5.0
Ease of Online booking	129893.0	2.756754	1.401660	0.0	2.0	3.0	4.0	5.0
Food and drink	129893.0	2.571501	1.568058	0.0	1.0	3.0	4.0	5.0
Seat comfort	129893.0	3.281093	1.528844	0.0	2.0	4.0	5.0	5.0
Inflight entertainment	129893.0	3.357794	1.334169	0.0	2.0	4.0	4.0	5.0
Baggage handling	129893.0	2.680822	1.628602	0.0	1.0	3.0	4.0	5.0
Checkin service	129893.0	2.642067	1.627489	0.0	1.0	3.0	4.0	5.0
Inflight service	129893.0	3.525971	1.323826	0.0	3.0	4.0	5.0	5.0
Cleanliness	129893.0	3.221590	1.458906	0.0	2.0	3.0	4.0	5.0

	count	unique	top	freq
Code	129896	129878	CD24961940	12
Gender	129894	2	Female	65914
Customer Type	129894	2	Loyal Customer	106108
State	59114	43	NJ	4587
Type of Travel	129894	2	Business travel	89696
Class	129894	3	Business	62164
satisfaction	129893	2	neutral or dissatisfied	73468

Yêu cầu:

Yêu cầu 3. Làm sạch và chuẩn bị dữ liệu (Preparation)

1. Kiểm tra và xử lý các dữ liệu thiếu trong Dataset (nếu có)
2. Kiểm tra và xử lý dữ liệu ngoại lai trong Dataset (nếu có)
3. Kiểm tra và xử lý dữ liệu trùng lặp trong Dataset (nếu có)
4. Chuyển đổi kiểu dữ liệu của các thuộc tính cho phù hợp (nếu cần)
5. Theo quy định của ngành hàng không, các chuyến bay có thời gian khởi hành trễ hơn 15 phút so với lịch bay được tính là chuyến bay trễ (Delay). Dựa trên cột thời gian trễ của chuyến bay tại sân bay khởi hành (Departure Delay in Minutes), tạo thêm một cột mới đặt tên là "Delay" bằng cách rời rạc hóa dữ liệu của cột Departure Delay in Minutes về dạng dữ liệu nhị phân; nếu < 15 thành giá trị No, ngược lại ≥ 15 thành Yes



Yêu cầu:



Yêu cầu 4: Lưu dữ liệu:

1. Lưu tập dữ liệu đã xử lý ở yêu cầu 3 ra file CSV: **Data_Airlines_OK.csv**
2. Lọc danh sách hành khách có mức độ hài lòng (thuộc tính satisfaction) là “neutral or dissatisfied” lưu ra file CSV: **Data_Airlines_Dissatisfied.csv**
3. Lọc lấy hành khách đi trên các chuyến bay trễ trong tập dữ liệu (Delay = Yes) nhưng vẫn hài lòng với chuyến bay (satisfaction = ‘satisfied’) lưu ra file CSV: **Data_Airlines_Delay_Satisfied.csv**



Q & A
Thank you!